

Tesoreria Distribuida

Manual tecnico de estudio

Equipo 18 – Proyecto Final de Sistemas Distribuidos

VERSION DE ESTUDIO – NO ENTREGAR

Documento interno para entender y poder explicar el sistema ante el profesor. **No** forma parte de la entrega oficial y puede parecerse a otra documentacion: es intencional, su unico fin es estudiar.

Mini banco distribuido en Java puro
Paquete `mx.ipn.escom.tesoreria`

Índice

1. Arquitectura general	2
1.1. El sistema en una pagina	2
1.2. Topologia: lider y replicas	2
1.3. Mapa de paquetes	3
1.4. Flujo de una transferencia de punta a punta	3
2. Arranque y configuracion del nodo	4
2.1. Build: fat jar con Maven	4
2.2. Punto de entrada: Node	5
2.3. Configuracion por entorno (TES_*)	6
2.4. Carga del dataset de cuentas	7
3. Servidor HTTP en NIO puro	7
3.1. Por que un servidor propio	8
3.2. Modelo de concurrencia: IoLoop y workers	8
3.3. Parseo de peticiones y rutas	9
3.4. Construccion de respuestas	10
4. Nucleo del banco: dinero, ledger e invariante	10
4.1. Dinero exacto en centavos	10
4.2. Cuentas y ledger	11
4.3. Transferencia atomica	12
4.4. Snapshot atomico del total (lectura consistente)	13
4.5. Invariante de saldo y numero de secuencia	14
5. Seguridad: JWT y credenciales	15
5.1. Emision y verificacion de JWT	15
5.2. Contraseñas y credenciales	15
5.3. Autenticador y proteccion de endpoints	16
6. API REST y endpoints	17
6.1. Mapa de rutas	17
6.2. Autenticacion: register y login	18
6.3. Operaciones de negocio: balance y transfer	19
6.4. Endpoints de monitoreo	20
7. Replicacion y reanudacion exacta por secuencia	20
7.1. Protocolo de replicacion	20
7.2. La replica: aplicar y seguir el flujo	22
7.3. Checkpoint local y reanudacion exacta	22
7.4. Por que no “borra y descarga”	23
8. Persistencia durable en Cloud Storage	23
8.1. La bitacora de transacciones	24
8.2. Acceso a Cloud Storage	25
8.2.1. Autenticacion con la cuenta de servicio	25
8.3. Recuperacion ante desastre total	26

9. Dashboard de observabilidad	27
9.1. La interfaz web	27
9.2. Datos de negocio	28
9.3. Métricas de infraestructura del SO	28
10. Generador de carga, pruebas y despliegue	29
10.1. Generador de carga	29
10.2. Harness de pruebas	30
10.3. Despliegue en GCP	31
10.4. Operacion el dia del concurso	32

1. Arquitectura general

1.1. El sistema en una pagina

Tesoreria Distribuida (Equipo 18) es un mini banco distribuido escrito en Java puro, sin frameworks web ni servidor de aplicaciones. El estado que mantiene es un *ledger* de cuentas (un libro de saldos) replicado entre tres nodos. Sobre ese ledger ofrece cuatro capacidades de cliente: registro de usuarios, *login* que emite un token JWT, consulta del saldo de una cuenta y transferencia de dinero entre dos cuentas. A esas operaciones se suman la **durabilidad** de la bitacora de transferencias en Cloud Storage (GCS) y un **dashboard** de monitoreo del cluster.

El paquete raiz es `mx.ipn.escom.tesoreria`. El build con Maven produce un *fat jar* ejecutable (un *jar-with-dependencies* cuya clase principal es `app/Node.java`). Una propiedad central del diseno es que *un mismo jar* arranca como lider o como replica segun la configuracion: el rol no se compila, se decide en tiempo de ejecucion leyendo variables de entorno con prefijo `TES_`. En concreto, `NodeConfig.isLeader()` considera al proceso lider cuando `TES_LEADER_HOST` esta vacio o ausente, y replica cuando esa variable nombra el host del lider a seguir. El secreto de firma de los tokens (`TES_JWT_SECRET`) debe tener el MISMO valor en los tres nodos para que un JWT emitido en cualquiera sea valido en todos.

Al arrancar, `Node.main` carga el dataset compartido de cuentas en un `core/Ledger.java` nuevo y lo envuelve en un `core/Bank.java`, que es quien posee la secuencia de *commits* y la difusion a observadores. Si la carga no produce cuentas, el nodo falla de inmediato en lugar de servir con un ledger vacio:

```
1 Ledger ledger = new Ledger();
2 int loaded = Dataset.loadInto(ledger, Path.of(config.datasetPath()), config.
    idBase());
3 if (loaded <= 0) {
4     throw new IllegalStateException("dataset loaded 0 accounts from "
5         + config.datasetPath() + "; refusing to start with an empty ledger")
6     ;
7 }
8 Bank bank = new Bank(ledger);
```

Listing 1: Node.main: carga del dataset y rechazo de un ledger vacio

1.2. Topologia: lider y replicas

El sistema corre en tres nodos. **nodo-1** es el **lider** (maquina `e2-standard-4`) y es el unico que recibe el trafico de clientes: sobre el se ejecutan registro, *login*, consulta de saldo y, sobre todo, las transferencias, porque es el lider quien aplica de verdad cada movimiento, le asigna un numero de secuencia y lo numera de forma monotona. **nodo-2** y **nodo-3** son **replicas** (maquinas `e2-standard-2`) que no originan transacciones de cliente: solo siguen el flujo de transferencias que el lider ya acepto y lo reapiplan localmente para converger al mismo estado.

Quien es lider y quien es replica lo decide la configuracion, no el codigo. `app/NodeConfig.java` lee `TES_LEADER_HOST` desde el entorno y expone el rol con `isLeader()`:

```
1 public boolean isLeader() {
2     return leaderHost == null || leaderHost.isBlank();
3 }
```

Listing 2: NodeConfig.isLeader: el rol se infiere de `TES_LEADER_HOST`

A partir de ese rol, `Node.main` ramifica el arranque. En el **lider** (cuando `config.isLeader()` es verdadero) corre, si GCS esta configurado, la recuperacion en frio de la bitacora con `durable/Journal.java`, y registra dos observadores sobre el banco: el propio `Journal`, que guarda de forma durable cada transferencia nueva, y un `cluster/ReplicaFeed.java`, un servidor TCP que transmite los *commits* en vivo hacia las replicas conectadas. En la **replica** (cualquier otro caso) se restaura primero

el *checkpoint* durable con `durable/ReplicaCheckpoint.java` si hay GCS, y luego se arranca un `cluster/ReplicaSync.java` que se pone al día desde el líder por número de secuencia y aplica los *commits* en vivo. El cliente pega al líder porque solo allí existe el camino que origina, valida y numera una transferencia; las réplicas son un seguidor convergente, no un punto de entrada de escritura.

1.3. Mapa de paquetes

El código se organiza por responsabilidad bajo `mx.ipn.escom.tesoreria`:

- **app** — arranque y configuración del proceso: `Node` (el main), `NodeConfig` (variables `TES_*`) y `NodeStats` (métricas del nodo).
- **net** — transporte HTTP sobre NIO crudo, sin librerías: `Server`, `IoLoop`, `Channel`, `RequestParser`, `Request`, `Reply`, `Routes` y la interfaz `Endpoint`.
- **core** — el dominio del banco: `Account`, `Ledger` (cuentas y dinero), `Bank` (orquesta la transacción y la secuencia), `Money`, `Transfer`, `TransferLog`, `TransferException`, `Dataset` y la interfaz `CommitListener`.
- **security** — autenticación: `Tokens` (JWT), `Passwords` (bcrypt), `Credential`, `CredentialStore` y `Authenticator`.
- **api** — los *endpoints* HTTP: `RegisterEndpoint`, `LoginEndpoint`, `BalanceEndpoint`, `TransferEndpoint`, `StatsEndpoint`, `PanelEndpoint` y `DashboardEndpoint`.
- **cluster** — replicación TCP (una línea JSON por transferencia): `ReplicaFeed` en el líder, `ReplicaSync` en la réplica y `WireCodec` para codificar el cable.
- **durable** — durabilidad en GCS por REST: `Journal`, `GcsStore`, `GcsAuth` y `ReplicaCheckpoint`.
- **loadtest** — `LoadDriver`, generador de carga y verificador de la invariante de conservación del dinero.

1.4. Flujo de una transferencia de punta a punta

Una transferencia recorre todas las capas en orden. Tomando como ejemplo `POST /api/transactions/transfer` sobre el líder:

1. **Cliente → red.** La petición llega al servidor NIO `net/Server.java`. El hilo aceptador la reparte por *round-robin* a un reactor (`net/IoLoop.java`); los reactores hacen la E/S y un pool de *workers* computa la respuesta.
2. **Ruteo.** `net/Routes.java` empareja método y ruta y despacha al *endpoint* registrado en `Node.main` para `POST /api/transactions/transfer`: una instancia de `api/TransferEndpoint.java`.
3. **Validación JWT.** El *endpoint* primero exige `POST` (devuelve 405 en otro caso) y luego autoriza el token con `security/Authenticator.java`; sin token válido responde 401.
4. **Parseo y conversión a centavos.** Lee del cuerpo JSON las claves obligatorias `sourceAccountId`, `targetAccountId` (ids como cadenas) y `amount` (decimal), convirtiendo el monto a centavos con `core/Money.java`. Un cuerpo malformado da 400 `bad_request`; un id numérico fuera del rango `int` da 404 `no_such_account`.
5. **Banco y ledger.** Entrega el movimiento a `Bank.transfer(from, to, cents)`, que rechaza por política la auto-transferencia y el monto no positivo, delega el débito/credito atómico al `core/Ledger.java`, y solo si tiene éxito estampa el número de secuencia, lo registra en el `core/TransferLog.java` y avanza los contadores.

6. **Difusion a observadores.** En el mismo `transfer`, el banco notifica a cada `CommitListener` registrado: el `durable/Journal.java`, que persiste la transferencia en la bitacora GCS, y el `cluster/ReplicaFeed.java`, que la empuja a las replicas conectadas.
7. **Feed a replicas.** Cada replica recibe el `commit` por su `cluster/ReplicaSync.java` y lo re replica con `Bank.applyReplicated`, que es idempotente por secuencia para no duplicar saldos tras una reconexion.
8. **Respuesta.** El `endpoint` devuelve 200 con `{"status":"ok","seq":<seq>}`; un rechazo del banco (`TransferException`) se mapea a 404 para `no_such_account` y a 400 para los demas codigos, con cuerpo `{"error":<code>}`.

El nucleo del paso 5, ya en `api/TransferEndpoint.java`, entrega el movimiento al banco y traduce el resultado a una respuesta REST:

```

1 try {
2     long seq = bank.transfer(from, to, cents);
3     JsonObject ok = new JsonObject();
4     ok.addProperty("status", "ok");
5     ok.addProperty("seq", Long.valueOf(seq));
6     return Reply.json(200, gson.toJson(ok));
7 } catch (TransferException e) {
8     int status = "no_such_account".equals(e.code()) ? 404 : 400;
9     JsonObject err = new JsonObject();
10    err.addProperty("error", e.code());
11    return Reply.json(status, gson.toJson(err));
12 }

```

Listing 3: `TransferEndpoint.serve`: del banco a la respuesta HTTP

2. Arranque y configuracion del nodo

2.1. Build: fat jar con Maven

El proyecto se compila con Maven y produce un *fat jar* ejecutable descrito en `pom.xml`. El artefacto es `tesoreria-distribuida` version 1.0.0, empaquetado como `jar` y compilado contra Java 17 (`maven.compiler.release = 17`). El comando habitual es `mvn clean package`, que dispara el plugin `maven-assembly-plugin` (version 3.7.1) con el descriptor `jar-with-dependencies`: ese descriptor reúne el código del nodo y todas sus dependencias en un solo `jar` autocontenido cuyo manifiesto fija la clase principal `mx.ipn.escom.tesoreria.app.Node`. Así un unico `jar` arranca indistintamente como líder o como replica, según las variables de entorno.

```

1 <artifactId>maven-assembly-plugin</artifactId>
2 <version>3.7.1</version>
3 <configuration>
4     <descriptorRefs>
5         <descriptorRef>jar-with-dependencies</descriptorRef>
6     </descriptorRefs>
7     <archive>
8         <manifest>
9             <mainClass>mx.ipn.escom.tesoreria.app.Node</mainClass>
10        </manifest>
11    </archive>
12 </configuration>

```

Listing 4: Plugin assembly: fat jar y clase principal (`pom.xml`)

El sistema es Java puro, sin frameworks: la lista de dependencias declaradas se limita a tres bibliotecas, `org.mindrot:jbcrypt:0.4` (hash de contraseñas), `com.auth0:java-jwt:4.3.0`

(emitir y validar JWT) y `com.google.code.gson:gson:2.13.1` (parseo de JSON). Es importante para la rubrica que el `pom.xml` *no* traiga ningun driver JDBC ni base de datos embebida: el requisito de usar un SGBD se cumple con cero, porque el estado vive en memoria (el ledger) y la durabilidad se resuelve contra Cloud Storage. De hecho el comentario del propio `pom.xml` aclara que no se permite el SDK de Google Cloud, y que GCS se alcanza por su API REST JSON cruda usando `java.net.http.HttpClient`.

2.2. Punto de entrada: Node

La clase `app/Node.java` es el punto de entrada del proceso. Su `main()` recibe opcionalmente el puerto HTTP como `args[0]` (por defecto 8080) y orquesta el arranque en una secuencia fija. Primero lee la configuracion con `NodeConfig.fromEnv()`; despues carga el dataset de cuentas en un Ledger nuevo con `Dataset.loadInto(ledger, ...)` y, si la carga devuelve cero cuentas, falla de inmediato con `IllegalStateException` en vez de servir un ledger vacio (sintoma tipico de un CSV ausente o truncado). El ledger ya poblado se envuelve en un `Bank`, que es quien posee la secuencia de *commits* y el reparto a los `CommitListener`.

```

1  int port = (args.length > 0) ? Integer.parseInt(args[0]) : 8080;
2
3  NodeConfig config = NodeConfig.fromEnv();
4
5  Ledger ledger = new Ledger();
6  int loaded = Dataset.loadInto(ledger, Path.of(config.datasetPath()), config.
    idBase());
7  if (loaded <= 0) {
8      throw new IllegalStateException("dataset loaded 0 accounts from "
9          + config.datasetPath() + "; refusing to start with an empty ledger")
10     ;
11 }
12 // ...
13 Bank bank = new Bank(ledger);

```

Listing 5: Inicio de `Node.main()`: config, dataset y bank

A continuacion el nodo se ramifica segun su rol, determinado por `config.isLeader()`. Si es **lider** y hay Cloud Storage configurado (`TES_GCS_KEYFILE` y `TES_BUCKET`), construye un `GcsStore` y un `Journal`, ejecuta la recuperacion en frio con `journal.recover(bank::applyReplicated)` para reaplicar las transferencias durables, registra el journal como `CommitListener` y agrega un *shutdown hook* que drena la cola durable al apagar. Tambien levanta el `ReplicaFeed` en el puerto de replicacion y lo registra como otro listener, de modo que cada commit en vivo se transmita por TCP a las replicas conectadas. Si es **replica**, opcionalmente restaura su `ReplicaCheckpoint` desde GCS *antes* de iniciar la replicacion (para que el `CATCHUP` arranque desde la marca exacta ya aplicada y no desde 0), y luego un `ReplicaSync` se pone al dia desde el lider y aplica los commits en vivo a traves del banco.

Por ultimo, `main()` arma la pila de seguridad (`Tokens`, `CredentialStore`, `Authenticator`), construye el proveedor de metricas `NodeStats`, registra la tabla de rutas con las rutas exactas del contrato y arranca el servidor. La ultima linea bloquea el hilo principal mientras los reactores y los workers atienden las peticiones.

```

1  Routes routes = new Routes();
2  routes.register("POST", "/api/register", new RegisterEndpoint(authenticator));
3  routes.register("POST", "/api/login", new LoginEndpoint(authenticator));
4  routes.register("GET", "/api/accounts/{id}", new BalanceEndpoint(authenticator,
    ledger));
5  routes.register("POST", "/api/transactions/transfer", new TransferEndpoint(
    authenticator, bank));
6  routes.register("GET", "/api/stats", new StatsEndpoint(stats));
7  routes.register("GET", "/panel", new PanelEndpoint(stats, config));

```

```

8 routes.register("GET", "/", new DashboardEndpoint());
9
10 Server server = new Server(port, routes, config.workers(), config.reactors());
11 server.start();
12
13 // Block the main thread while the IoLoop and workers serve requests.
14 Thread.currentThread().join();

```

Listing 6: Tabla de rutas y arranque del servidor (Node.main())

El proveedor de metricas se instancia como `NodeStats(ledger, bank, journal, config)` (ver `app/NodeStats.java`): toma el numero de cuentas y el saldo total del `Ledger`, el conteo de transferencias y el ultimo id del `Bank`, y el conteo en GCS del `Journal` (que vale 0 en las replicas, donde el `journal` es null). Las metricas de host (CPU, RAM, disco) se leen con Java crudo desde `/proc/stat`, `/proc/meminfo` y el sistema de archivos, sin bibliotecas externas.

2.3. Configuración por entorno (TES_*)

Toda la configuración del nodo se inyecta por variables de entorno con prefijo `TES_`, leídas una sola vez en `NodeConfig.fromEnv()` dentro de `app/NodeConfig.java`. El rol se deriva de `TES_LEADER_HOST`: vacío o ausente significa que el proceso es el **lider** (nodo-1); cualquier otro valor nombra al host del lider que esta **replica** debe seguir. El secreto `TES_JWT_SECRET` debe ser el MISMO en los tres nodos para que un token emitido en cualquiera valide en todos. La siguiente tabla resume cada variable con su significado y su valor por defecto (los defaults vienen de las constantes `DEFAULT_*` de la clase).

Variable	Default	Significado
<code>TES_DATASET</code>	(ninguno)	Ruta al CSV de cuentas que carga el nodo.
<code>TES_JWT_SECRET</code>	(ninguno)	Secreto HMAC para firmar/validar JWT; identico en los 3 nodos.
<code>TES_NODE_ID</code>	(ninguno)	Identificador legible del nodo, p.ej. nodo-1 .
<code>TES_PEERS</code>	""	URLs base de los pares, separadas por coma, para la agregacion del panel.
<code>TES_LEADER_HOST</code>	""	Host del lider a seguir; vacío = este nodo ES el lider.
<code>TES_REPL_PORT</code>	9090	Puerto TCP del feed de replicacion del lider.
<code>TES_BUCKET</code>	(ninguno)	Bucket de Cloud Storage para el <code>journal</code> durable.
<code>TES_GCS_KEYFILE</code>	(ninguno)	Ruta al key file de la cuenta de servicio para autenticar contra GCS.
<code>TES_WORKERS</code>	16	Tamano del pool de hilos que procesan peticiones.
<code>TES_ID_BASE</code>	1	Id asignado a la primera cuenta del dataset.
<code>TES_REPLICA_CHECKPOINT_INTERVAL_SECS</code>	10	Segundos entre flushes del checkpoint de la replica a GCS.
<code>TES_REPLICA_CHECKPOINT_PATH</code>	(deriva)	Nombre del objeto del checkpoint; si es null usa <code>checkpoint/<nodeId>.json</code> .
<code>TES_REACTORS</code>	1 por CPU	Numero de reactores (selectores) del servidor HTTP.

La lectura usa dos ayudantes privados: `env(nombre, def)` devuelve el default cuando la variable esta ausente o vacia, e `intEnv(nombre, def)` parsea un entero y cae al default si la variable falta o no es numerica. El metodo `isLeader()` centraliza la decision de rol: devuelve `true` cuando `leaderHost` es null, vacio o solo espacios en blanco.

```

1 public static NodeConfig fromEnv() {
2     return new NodeConfig(
3         env("TES_DATASET", null),
4         env("TES_JWT_SECRET", null),
5         env("TES_NODE_ID", null),
6         env("TES_PEERS", ""),
7         env("TES_LEADER_HOST", ""),
8         intEnv("TES_REPL_PORT", DEFAULT_REPL_PORT),
9         // ...
10        intEnv("TES_WORKERS", DEFAULT_WORKERS),
11        intEnv("TES_ID_BASE", DEFAULT_ID_BASE),
12        // ...
13        intEnv("TES_REACTORS", DEFAULT_REACTORS));
14 }

```

Listing 7: Lectura de las variables TES_* en NodeConfig.fromEnv()

2.4. Carga del dataset de cuentas

Las cuentas se cargan desde el CSV con `Dataset.loadInto()` en `core/Dataset.java`. El archivo de referencia es `material-profesor/alumnos.csv`, entregado por el profesor y consumido tal cual (no se regenera ni se edita). Es CSV plano de cuatro columnas y *sin* fila de encabezado, con el formato `nombre,apellido1,apellido2,saldo`; por eso la primera linea tambien se parsea, no se salta. Como no hay columna de id, el id de cada cuenta es su posicion en el archivo contando desde `idBase` (normalmente 1, via `TES_ID_BASE`): la primera linea de datos pasa a ser la cuenta `idBase`, la segunda `idBase+1`, y asi sucesivamente. Que todos los nodos lean el mismo archivo es justo lo que hace significativa la verificacion de conservacion del dinero en el cluster.

El nombre del titular es la union con espacios de los tres campos de nombre. El saldo se convierte de su texto decimal a centavos enteros con `Money.toCents(String)`, de modo que ningun punto flotante toque jamas un saldo: los importes se manejan internamente en centavos. Las lineas vacias o con menos de cuatro columnas se descartan, y el metodo devuelve el numero de cuentas efectivamente cargadas (el valor que `Node.main()` exige que sea mayor que cero).

```

1 int id = idBase;
2 int loaded = 0;
3 // ...
4 String[] cols = line.split(",", -1);
5 if (cols.length < 4) {
6     continue;
7 }
8 String owner = cols[0].trim() + " " + cols[1].trim() + " " + cols[2].trim();
9 long cents = Money.toCents(cols[3].trim());
10 ledger.add(new Account(id, owner, cents));
11 id++;
12 loaded++;

```

Listing 8: Asignacion de id por indice de fila y saldo en centavos (`Dataset.loadInto`)

3. Servidor HTTP en NIO puro

El transporte HTTP de la tesoreria no se apoya en `com.sun.net.httpserver` ni en ningun framework: es una pila propia escrita sobre `java.nio`, repartida en ocho clases del paquete `net`.

`net/Server.java` es la puerta de entrada, `net/IoLoop.java` es el reactor que hace toda la E/S, `net/Channel.java` guarda el estado de cada conexion, `net/RequestParser.java` parsea HTTP de forma incremental, `net/Request.java` y `net/Reply.java` modelan peticion y respuesta, y `net/Routes.java` con `net/Endpoint.java` resuelven el ruteo. Este capitulo explica por que se construyo asi y como encajan las piezas.

3.1. Por que un servidor propio

La razon principal es doble. Primero, la regla de "Java puro" del concurso prohíbe servidores de libreria; escribir el transporte sobre `java.nio` mantiene la dependencia unicamente en el JDK. Segundo, y mas importante, el concurso evalua bajo carga, y un servidor propio da control fino sobre la concurrencia: se decide cuantos reactores corren, cuantos hilos de trabajo computan respuestas y como se reparte la E/S entre nucleos.

Un servidor de libreria basado en un `ThreadPool` bloqueante (un hilo por conexion) gasta un hilo del sistema operativo por cada socket abierto, lo que no escala cuando hay miles de conexiones en keep-alive mayormente ociosas. El diseno NIO multiplexa muchas conexiones sobre pocos hilos mediante un `Selector`: un solo hilo reactor vigila cientos de sockets y solo despierta para los que tienen datos listos. Asi se separa la E/S (barata, dirigida por eventos) del computo de la respuesta (que puede tocar el banco y la replicacion), y cada parte se dimensiona por separado.

3.2. Modelo de concurrencia: IoLoop y workers

El diseno sigue el patron Reactor con pool de trabajadores (Doug Lea, patron 3) y usa tres clases de hilo con responsabilidades disjuntas, lo que evita casi todo bloqueo compartido.

El `Server` posee un `ServerSocketChannel`, un hilo aceptador dedicado, un arreglo de reactores `IoLoop` (cada uno con su propio `Selector`) y un pool compartido de hilos worker. El numero de reactores y de workers se fija en el constructor; en produccion los provee `app/NodeConfig.java` desde las variables de entorno `TES_REACTORS` (por defecto uno por CPU) y `TES_WORKERS` (por defecto 16). Con `reactors == 1` se obtiene el reactor unico clasico, un respaldo seguro.

El **hilo aceptador** corre `acceptLoop()`: se bloquea en `accept()` y reparte cada conexion nueva entre los reactores en round-robin. Repartir aqui evita que un solo reactor sature un nucleo mientras los demas estan ociosos, que es el cuello de botella del reactor unico.

```
1 SocketChannel socket = serverChannel.accept();
2 if (socket == null) {
3     continue;
4 }
5 socket.configureBlocking(false);
6 loops[next].register(socket);
7 next = (next + 1) % loops.length;
```

Listing 9: El aceptador reparte cada conexion en round-robin (`net/Server.java`)

Cada **hilo reactor** (`IoLoop`) hace solo lectura y escritura para sus conexiones; nunca calcula una `Reply` en linea. Su `run()` se bloquea en `selector.select()` y, al despertar, primero drena dos colas: `registerPending()` registra los sockets que el aceptador dejo en cola, y `enableQueuedWrites()` activa `OP_WRITE` para las llaves cuya respuesta ya esta lista. Esto es deliberado: el registro de canales y las mutaciones del conjunto de intereses (`interestOps`) ocurren unicamente en el hilo del reactor, de modo que el aceptador y los workers se comunican mediante colas concurrentes (`ConcurrentLinkedQueue`) y `selector.wakeup()`, manteniendo fuera del diseno la clasica condicion de carrera entre hilos sobre el `Selector`.

Cuando `onRead()` lee bytes y el parser reporta una peticion completa, el reactor no la atiende: pausa las lecturas de esa conexion con `key.interestOps(0)` y envia el trabajo al pool con `workers.submit(...)`. Pausar las lecturas garantiza una sola peticion en vuelo por conexion, lo que ordena las respuestas sin necesidad de pipelining.

```

1 boolean complete = channel.parser().feed(buffer);
2 buffer.compact();
3 if (complete) {
4     Request request = channel.parser().take();
5     channel.setProcessing(true);
6     // Pause reads until this reply is flushed: one request in flight per
7     // connection, which keeps responses ordered without pipelining.
8     key.interestOps(0);
9     workers.submit(() -> handle(key, channel, request));
10 }

```

Listing 10: El reactor delega el computo al worker (net/IoLoop.java)

El hilo **worker** ejecuta `handle()`: rutea con `routes.match(...)`, invoca al `Endpoint`, serializa la `Reply` a un `ByteBuffer` HTTP/1.1 y la encola en el `Channel`. Luego devuelve la llave al reactor anadiendola a `writeReady` y llamando `selector.wakeup()`; el reactor activara `OP_WRITE` y `onWrite()` hara el flush real. El estado por conexion vive en `Channel`, cuyo punto de encuentro es la cola `outbound`: el worker la llena con `enqueue()` (sincronizado) y el reactor la vacia con `peekOutbound()/popOutbound()` en cada evento de escritura. Si el buffer de envio del socket se llena, `onWrite()` deja la escritura a medias y conserva el interes en `OP_WRITE` para reintentar despues.

3.3. Parseo de peticiones y rutas

Cada `Channel` tiene su propio `RequestParser`, un parser HTTP/1.1 incremental con maquina de estados explicita (`READ_LINE`, `READ_HEADERS`, `READ_BODY`, `DONE`). Su metodo `feed(ByteBuffer)` consume los bytes conforme llegan, sin re-escanear el buffer desde cero, y devuelve `true` cuando hay una peticion completa disponible via `take()`. En `READ_LINE` y `READ_HEADERS` acumula una linea terminada en `CRLF`; al llegar el `\n`, `parseRequestLine()` divide "METHOD destino HTTP/x.y" (separando el destino en `path` y `query` por el primer `?`) y `parseHeader()` guarda cada cabecera con su nombre en minusculas. La linea vacia cierra las cabeceras: si hay `Content-Length` se pasa a `READ_BODY` y se leen exactamente esos bytes; si no, la peticion ya esta completa. `take()` entrega un `Request` inmutable y resetea la maquina para la siguiente peticion sobre la misma conexion keep-alive. El parser tambien decide el keep-alive (`computeKeepAlive()`): respeta la cabecera `Connection` y, en su ausencia, mantiene viva la conexion solo en HTTP/1.1.

El `Request` resultante es una vista inmutable: expone `method()`, `path()`, `query()`, `header(name)` (insensible a mayusculas), `body()/bodyText()` y `pathParam(name)`. La firma del handler la define `net/Endpoint.java`, una interfaz funcional con un solo metodo:

```

1 @FunctionalInterface
2 public interface Endpoint {
3
4     /** Handles one request and produces its reply. */
5     Reply serve(Request req);
6 }

```

Listing 11: Contrato de un manejador de peticiones (net/Endpoint.java)

El ruteo lo hace `Routes`, un router por metodo y path. Cada patron se parte en segmentos por `/`; un segmento escrito como `{name}` es una variable de un solo segmento y los demas deben coincidir por igualdad exacta de cadena (es coincidencia exacta de segmentos, no de prefijo mas largo). `register(method, pattern, endpoint)` compila el patron y `match(method, path)` lo resuelve, capturando las variables. Las semanticas de resolucion son precisas: si coinciden el metodo y la forma de segmentos devuelve un `Match` con estado 200 y el `Endpoint`; si algun patron coincide con el path pero ningun metodo encaja devuelve 405; y si ningun patron coincide, 404.

```

1 for (int i = 0; i < segments.length; i++) {

```

```

2   String pat = route.segments[i];
3   if (isVariable(pat)) {
4       params.put(pat.substring(1, pat.length() - 1), segments[i]);
5   } else if (!pat.equals(segments[i])) {
6       shapeMatches = false;
7       break;
8   }
9 }

```

Listing 12: Coincidencia de segmentos y captura de variables (net/Routes.java)

En `IoLoop.handle()`, cuando `match.status` es 200 se llama `request.bindPathParams(match.params)` para inyectar las variables capturadas y luego `match.endpoint.serve(request)`. Un estado distinto de 200 se traduce a `Reply.status(match.status)` (404 o 405); si el endpoint devuelve null o lanza una `RuntimeException`, el reactor responde 500.

3.4. Construcción de respuestas

La respuesta se modela con `net/Reply.java`, un objeto que el worker construye y el reactor serializa después. Lleva el código de estado, el `Content-Type` y el cuerpo en bytes. Sus factorías estándar cubren los tipos que emite el servicio: `Reply.json(status, json)` fija `application/json`; `charset=utf-8` y codifica el cuerpo en UTF-8; `Reply.text(...)` produce `text/plain`; `Reply.html(...)` sirve `text/html` a partir de bytes crudos (por ejemplo una página leída de recursos); y `Reply.status(int)` arma una respuesta solo de estado con cuerpo vacío, la que usa el reactor para 404, 405 y 500.

```

1 public static Reply json(int status, String json) {
2     return new Reply(status, "application/json; charset=utf-8",
3         json.getBytes(StandardCharsets.UTF_8));
4 }

```

Listing 13: Factoría JSON de respuestas (net/Reply.java)

`Reply` ofrece además `close()`, que marca la conexión para cerrarse tras enviar la respuesta (encadenable porque devuelve `this`). La escritura efectiva del HTTP no ocurre en `Reply`: el reactor la serializa en `IoLoop.serialize()`, que arma la línea de estado HTTP/1.1 `<código> <razón>` con las cabeceras `Content-Type`, `Content-Length` y `Connection` (`keep-alive` o `close`, según lo decidido entre `request.wantsClose()` y `reply.isClose()`), seguidas del cuerpo. La razón textual la traduce `reason(int)` para los códigos que el servicio emite (200, 201, 400, 401, 403, 404, 405, 409, 500). De este modo `Reply` permanece como un valor inmutable y agnóstico al cable, y toda la mecánica de bytes y `keep-alive` queda concentrada en el reactor.

4. Núcleo del banco: dinero, ledger e invariante

El paquete `core` concentra el dominio del mini-banco: como se representa el dinero, donde viven los saldos, como se mueve dinero de una cuenta a otra sin condiciones de carrera y que invariante debe respetarse siempre. Es el corazón de la rubrica de Consistencia (25 puntos): el saldo global debe conservarse, sin inconsistencias, aun bajo carga concurrente. Todo lo que sigue se apoya en una decisión de diseño simple pero estricta: el dinero nunca se representa con punto flotante.

4.1. Dinero exacto en centavos

Un saldo jamás se guarda como `double`: se guarda como un entero de centavos (`long`). Así se evitan los errores de redondeo binario típicos del punto flotante, donde una cantidad como 0.10 no tiene representación exacta y las sumas acumulan deriva. Al trabajar en centavos enteros, toda operación aritmética (sumar, restar, comparar) es exacta y reproducible en cualquier nodo.

Por eso los saldos viven como `long` en `core/Account.java` (campo `balanceCents`) y los montos de cada transferencia tambien, en el campo `cents` de `core/Transfer.java`. La clase `core/Money.java` no almacena dinero ni hace aritmetica de saldos: es un ayudante estatico (constructor privado, sin instancias) que centraliza la unica conversion entre la cadena decimal que ve el humano (con dos decimales) y la representacion interna en centavos. Se extrajo a proposito para que los endpoints, el cargador de datos y el codec del cluster nunca incrusten aritmetica de `BigDecimal` ad hoc.

La conversion de texto a centavos usa `BigDecimal` con redondeo `HALF_UP`, de modo que el parseo nunca introduce error de coma flotante:

```
1 public static long toCents(String decimal) {
2     return new BigDecimal(decimal)
3         .movePointRight(2)
4         .setScale(0, RoundingMode.HALF_UP)
5         .longValueExact();
6 }
```

Listing 14: Conversion exacta de decimal a centavos en `Money.toCents`

El camino inverso, `Money.toDecimal(long)`, reconstruye la cadena con exactamente dos decimales usando `movePointLeft(2)` y `setScale(2)`. La aritmetica real de mover dinero (la resta del origen y la suma al destino) no ocurre en `Money`, sino en el ledger, sobre esos mismos `long` de centavos.

4.2. Cuentas y ledger

Una `core/Account.java` es una cuenta del banco: su id y su titular (`owner`) son inmutables, y el unico estado mutable es el saldo en centavos. Ese saldo solo puede leerse o escribirse manteniendo el monitor intrinseco de la propia cuenta; por eso tanto el lector `balanceCents()` como el escritor `setBalanceCents(long)` estan declarados `synchronized`. No hay un objeto candado por campo: el ledger toma los monitores intrinsecos de las dos cuentas que participan en una transferencia, ordenados por id, para mantener el protocolo libre de deadlock.

El `core/Ledger.java` guarda el conjunto de cuentas y realiza la mecanica cruda de mover dinero entre dos de ellas; deliberadamente no sabe nada de secuencias, log de commits ni replicacion (eso vive en `Bank`). Las cuentas se almacenan en un `ConcurrentHashMap<Integer, Account>`, de modo que las lecturas (`get`) y el alta de cuentas (`add`) son libres de candado. La consulta de un saldo individual es simplemente `ledger.get(id).balanceCents()`. La suma global se obtiene con `totalCents()`, que recorre todas las cuentas acumulando sus saldos como una lectura consistente: aun bajo movimientos concurrentes ese total es exactamente el valor cargado al inicio, porque el escaneo toma un snapshot atomico (lo detalla la subseccion *Snapshot atomico del total*). Observarlo bajo carga es la forma en que el sistema verifica que las transferencias conservan el dinero.

Todos los nodos parten del mismo estado gracias a `core/Dataset.java`, que carga el dataset compartido de cuentas en un ledger. El archivo es un CSV plano de cuatro columnas sin encabezado (`firstName,lastName1,lastName2,balance`) y sin columna de id: el id de cada cuenta es su posicion en el archivo contando desde `idBase` (normalmente 1), asi que la primera linea de datos se parsea, no se omite. El titular se forma uniando los tres campos de nombre con espacios, y el saldo se convierte de su texto decimal a centavos enteros con `Money.toCents`, de modo que ningun punto flotante toca jamas un saldo. Como cada nodo lee el mismo archivo, todos arrancan con saldos identicos, y por eso la comprobacion de conservacion de dinero a lo largo del cluster es significativa.

4.3. Transferencia atomica

La orquestacion de una transferencia solicitada por el cliente vive en `Bank.transfer(int, int, long)` de `core/Bank.java`. El banco primero aplica las validaciones que son politica del banco (no mecanica del ledger): rechaza una transferencia a la misma cuenta y un monto no positivo. Solo si esas comprobaciones pasan, delega el debito y el credito al ledger y, en el mismo paso atomico (dentro del candado del movimiento), le asigna el siguiente numero de secuencia; luego la registra en el log y la entrega a los listeners:

```

1 public long transfer(int from, int to, long cents) throws TransferException {
2     if (from == to) {
3         throw TransferException.selfTransfer();
4     }
5     if (cents <= 0L) {
6         throw TransferException.badAmount();
7     }
8     // La secuencia se asigna DENTRO del candado del movimiento (atomica con el)
9     long seq = ledger.move(from, to, cents, sequence::incrementAndGet);
10    Transfer committed = new Transfer(seq, from, to, cents);
11    // ...
12 }

```

Listing 15: Bank.transfer: politica, movimiento delegado y commit

La parte critica para la consistencia, la atomicidad y la ausencia de deadlock, no esta en `Bank` sino en el metodo privado `apply` del ledger, al que llega `ledger.move`. Ese metodo busca ambas cuentas y, para mover el dinero como un solo paso atomico, toma los dos monitores en un orden fijo determinado por el id (siempre el menor primero). Ese orden global es lo que impide el deadlock: si dos transferencias inversas tocan el mismo par de cuentas en sentidos opuestos, ambas intentan adquirir los candados en la misma secuencia, de modo que nunca se bloquean mutuamente esperando candados cruzados.

```

1 Account first = (from < to) ? src : dst;
2 Account second = (from < to) ? dst : src;
3 synchronized (first) {
4     synchronized (second) {
5         if (enforceFunds && src.balanceCents() < cents) {
6             throw TransferException.lowBalance(from);
7         }
8         src.setBalanceCents(src.balanceCents() - cents);
9         dst.setBalanceCents(dst.balanceCents() + cents);
10        // Estampa la secuencia aqui dentro: el orden de commit coincide
11        // con el orden en que se movio el dinero.
12        return (sequencer != null) ? sequencer.getAsLong() : 0L;
13    }
14 }

```

Listing 16: Ledger.apply: orden de candados por id y movimiento atomico

Como ambos monitores se mantienen durante todo el movimiento, el par de saldos cambia junto, sin que otro hilo observe un estado intermedio donde el dinero ya salio del origen pero aun no entro al destino. El rechazo nunca se devuelve como valor de retorno, sino como una `core/TransferException.java`, una excepcion verificada con un `code()` de un vocabulario pequeno y cerrado: `self_transfer`, `bad_amount`, `no_such_account` y `low_balance`. Asi un llamador no puede continuar por accidente despues de un movimiento rechazado, y la capa HTTP mapea esos codigos a estados y a un cuerpo JSON de error. La `core/Transfer.java` resultante es un `record` inmutable (`seq, from, to, cents`): es la unidad de replicacion y durabilidad que el codec del cluster y el journal en GCS serializan.

4.4. Snapshot atomico del total (lectura consistente)

Sumar las 820k cuentas mientras el banco recibe transferencias no es inocuo: un escaneo ingenuo puede caer justo entre el debito y el credito de una misma transferencia (origen ya restado, destino aun no sumado) y reportar un total *roto* que no corresponde a ningun estado real. Para que `Ledger.totalCents()` sea siempre una lectura consistente, el ledger anade un segundo mecanismo de sincronizacion, separado de los monitores de cuenta: un `ReentrantReadWriteLock` llamado `snapshotLock`.

La clave es una inversion deliberada de los lados del candado. Cada movimiento de dinero (los caminos `move`, `settle` y el `apply` privado que comparten) toma el lado **COMPARTIDO** (read) de `snapshotLock` mientras hace su debito y su credito; el escaneo de `totalCents()` toma el lado **EXCLUSIVO** (write). Es decir, los lectores-del-candado son los escritores-del-dinero, y el escritor-del-candado es el unico lector global. Como el lado compartido admite cualquier numero de titulares simultaneos, todas las transferencias siguen ejecutandose en paralelo entre si; pero el lado exclusivo excluye a todas, de modo que la suma no puede solaparse jamas con una transferencia a medio aplicar. El escaneo observa, por construccion, un estado donde cada transferencia esta completa o aun no empezada: un *snapshot* atomico.

```

1 // En apply(...): el debito+credito corre bajo el lado COMPARTIDO,
2 // asi muchas transferencias avanzan a la vez. El readLock se toma ANTES
3 // de los monitores de cuenta (el monitor de cuenta es el sumidero del grafo).
4 snapshotLock.readLock().lock();
5 try {
6     synchronized (first) {
7         synchronized (second) {
8             src.setBalanceCents(src.balanceCents() - cents);
9             dst.setBalanceCents(dst.balanceCents() + cents);
10            return (sequencer != null) ? sequencer.getAsLong() : 0L;
11        }
12    }
13 } finally {
14     snapshotLock.readLock().unlock();
15 }
16
17 // En totalCents(): el escaneo corre bajo el lado EXCLUSIVO, que excluye
18 // a todo movimiento, por lo que nunca ve un debito sin su credito.
19 public long totalCents() {
20     snapshotLock.writeLock().lock();
21     try {
22         long sum = 0L;
23         for (Account a : accounts.values()) {
24             sum += a.balanceCents();
25         }
26         return sum;
27     } finally {
28         snapshotLock.writeLock().unlock();
29     }
30 }

```

Listing 17: Ledger: lado compartido en el movimiento, lado exclusivo en el escaneo

Las dos capas de candados componen sin riesgo de deadlock porque forman un grafo dirigido aciclico: el `readLock` se adquiere *antes* que los monitores de cuenta, y el monitor de cuenta es siempre el sumidero (nunca se toma el `snapshotLock` teniendo ya un monitor de cuenta). Asi, el orden por id que ya evitaba el deadlock entre transferencias inversas se mantiene intacto, y el nuevo candado solo agrega un nivel por encima.

Por que el invariante queda exacto en los tres nodos. Como toda transferencia es suma-cero (un debito acompañado de un credito igual), el total verdadero es invariante: no cambia jamas. Y como el escaneo de cada nodo corre bajo el lado exclusivo, no puede capturar

una transferencia a medio aplicar; por tanto reporta exactamente ese total invariante, no un valor transitorio. Los tres nodos parten del mismo dataset, de modo que los tres escaneos atomicos devuelven la misma constante aun bajo carga. Eso es justo lo que el sello de consistencia del dashboard compara: con el snapshot atomico, el sello permanece verde bajo trafico sostenido en lugar de parpadear por lecturas rotas espurias.

Por que fair evita la inanicion del escaneo. El `snapshotLock` se construye en modo justo (`new ReentrantReadWriteLock(true)`). Sin justicia, un flujo continuo de transferencias (lectores-del-candado) podria mantener el lado compartido siempre tomado y *matar de hambre* al escaneo (escritor-del-candado), que nunca conseguiria el lado exclusivo. Con justicia, el escritor que espera se atiende en orden de llegada, asi que el endpoint `/panel` no puede colgarse bajo carga sostenida de transferencias.

4.5. Invariante de saldo y numero de secuencia

El invariante central es la conservacion del dinero: como cada movimiento es un debito en una cuenta acompanado de un credito por la misma cantidad en otra, la suma global de saldos permanece constante. Ese es precisamente el valor que expone `Ledger.totalCents()`, leído como un snapshot atomico, y su constancia bajo carga es lo que demuestra que las transferencias no crean ni destruyen dinero. El ledger se mantiene a proposito estrecho (solo aplica debito y credito equilibrados) para que ese invariante sea facil de razonar.

El numero de secuencia es la otra mitad de la consistencia, y no vive en el ledger sino en `core/Bank.java`, como un `AtomicLong sequence`. Es monotono y solo avanza en un commit real: la secuencia (`sequence.incrementAndGet()`) se asigna *dentro* del candado del movimiento, atomica con el debito y el credito, de modo que una transferencia rechazada nunca consume un numero y, ademas, el orden de las secuencias coincide con el orden en que el dinero se movio. Eso importa para las replicas: como sirven lecturas mientras reproducen en orden de secuencia, estampar la secuencia junto al movimiento evita que una cuenta se vea *transitoriamente* en negativo (un debito reproducido antes del credito que lo fondea). Esa secuencia, estampada en cada `Transfer`, es la base de la replicacion y de la verificacion de consistencia: ordena los commits y permite a una replica saber hasta donde esta al dia. En el camino de replica, `Bank.applyReplicated(Transfer)` no genera secuencia nueva sino que la recibe con la entrada y eleva los contadores con `accumulateAndGet(t.seq(), Math::max)`, de modo que reaplicar una entrada duplicada nunca retrocede el avance.

Cada commit aceptado se registra en el `core/TransferLog.java`, un historial append-only y thread-safe. Conserva la secuencia ordenada de commits para que la alimentacion de replicas pueda reproducir el historial de catch-up a una replica recién conectada. Las altas ocurren en el camino de commit a la tasa plena de transferencias (20 % del trafico del concurso), asi que la lista interna es un `ArrayList` protegido por el monitor del objeto: anexar es $O(1)$ amortizado. Una lista copy-on-write seria un error aquí porque copiaria todo el arreglo en cada anexion, volviendo la carga sostenida un costo de copia cuadrático. Los escaneos de catch-up (`since`) son raros (solo cuando una replica se conecta) y devuelven una copia fresca de la cola que coincide, de modo que el llamador la itera sin sostener el candado.

Por ultimo, `core/CommitListener.java` es la interfaz funcional que notifica un commit. Los listeners se registran con `Bank.addCommitListener` y, en el código, el fan-out ocurre únicamente en el camino de cliente del líder: al final de `Bank.transfer` se recorre la lista (un `CopyOnWriteArrayList`, para no bloquear el registro) y se invoca `onCommit(committed)` en cada uno, en orden de registro. Los listeners concretos son el journal en GCS (durabilidad) y la alimentacion de replicas (replicacion en vivo). La via `applyReplicated` no vuelve a notificar a los listeners, porque ya esta consumiendo el flujo del líder y no debe re-emitirlo.

5. Seguridad: JWT y credenciales

La autenticación del API se apoya en tres piezas que viven en el paquete `security/`: un emisor/verificador de JWT (`security/Tokens.java`), un hashing de contraseñas (`security/Passwords.java`) con su almacén de credenciales (`security/Credential.java` y `security/CredentialStore.java`), y un orquestador (`security/Authenticator.java`) que une todo y protege los endpoints. El JWT es real (firmado con HMAC y verificado en cada petición protegida), no decorativo: el login valida la credencial y el registro guarda el hash, nunca la contraseña en claro.

5.1. Emision y verificacion de JWT

`security/Tokens.java` es un servicio de instancia que envuelve la librería `com.auth0.jwt` (`java-jwt`). Se construye con el secreto HMAC compartido y, a partir de él, crea un `Algorithm.HMAC256(secret)` y un `JWTVerifier` ya fijado al emisor `tesoreria`. Construir el verificador una sola vez en el constructor evita recompilar las reglas de verificación en cada petición.

```
1 public Tokens(String secret) {
2     this.algorithm = Algorithm.HMAC256(secret);
3     this.verifier = JWT.require(algorithm).withIssuer(ISSUER).build();
4 }
5
6 public String issue(String subject) {
7     return JWT.create()
8         .withIssuer(ISSUER)
9         .withSubject(subject)
10        .sign(algorithm);
11 }
12
13 public String validate(String token) {
14     DecodedJWT decoded = verifier.verify(token);
15     return decoded.getSubject();
16 }
```

Listing 18: Tokens: constructor, emision y validacion

Qué claims lleva. El método `issue(subject)` firma un token compacto con dos claims: el emisor (`iss = tesoreria`) y el sujeto (`sub`), que es el nombre de usuario autenticado. El token no incluye un claim de expiración (`exp`): no se llama a `withExpiresAt`, de modo que el token es válido mientras lo sea la firma. La firma es HMAC-SHA256 sobre el secreto compartido, lo que garantiza integridad y autenticidad pero no confidencialidad (el payload va en Base64, no cifrado), por lo que no debe guardarse información sensible en los claims.

Cómo se verifica. En cada petición protegida, `validate(token)` llama a `verifier.verify(token)`, que comprueba la firma HMAC256 y que el emisor sea `tesoreria`; si algo no cuadra, `java-jwt` lanza una `JWTVerificationException` (subtipo de `RuntimeException`). Si la verificación pasa, devuelve el sujeto, es decir, la identidad que viajaba en el token.

Secreto compartido entre nodos. El secreto se inyecta desde fuera: en `app/Node.java` se construye `new Tokens(config.jwtSecret())`, y ese valor proviene de la variable de entorno `TES_JWT_SECRET` leída por `app/NodeConfig.java`. Como documenta ese archivo, `TES_JWT_SECRET` debe tener *el mismo valor en los tres nodos*, de forma que un token emitido en cualquier nodo (por ejemplo, tras hacer login en el líder) valide en todos. Es la propiedad clave de un secreto HMAC simétrico: quien conoce el secreto puede tanto firmar como verificar.

5.2. Contraseñas y credenciales

Las contraseñas nunca se almacenan en texto plano. `security/Passwords.java` es un ayudante sin estado, con dos métodos estáticos respaldados por `JBCrypt` (`org.mindrot.jbcrypt.BCrypt`), que esconde la generación de la sal y la comparación tras una API mínima.

```
1 public static String encode(String raw) {  
2     return BCrypt.hashpw(raw, BCrypt.gensalt());  
3 }  
4  
5 public static boolean matches(String raw, String hash) {  
6     return BCrypt.checkpw(raw, hash);  
7 }
```

Listing 19: Passwords: hash y verificación con BCrypt

`encode(raw)` genera una sal nueva con `BCrypt.gensalt()` y produce el hash BCrypt, que ya lleva la sal incrustada en su cadena. Se usa BCrypt en lugar de un hash simple porque es deliberadamente lento (factor de coste) y salado por diseño, lo que dificulta los ataques de fuerza bruta y de tablas precalculadas. `matches(raw, hash)` re-deriva el hash del candidato con la misma sal del hash guardado y compara; `BCrypt.checkpw` encapsula esa comparación. Como cada llamada a `encode` usa una sal distinta, dos usuarios con la misma contraseña obtienen hashes diferentes.

Una credencial almacenada se modela con `security/Credential.java`, un record inmutable que empareja el `username` con su `passwordHash` BCrypt y, por contrato, nunca contiene la contraseña en claro:

```
1 public record Credential(String username, String passwordHash) {  
2 }
```

Listing 20: Credential: record inmutable

El registro de credenciales vive en `security/CredentialStore.java`, un almacén en memoria seguro para hilos basado en un `ConcurrentHashMap<String, Credential>` indexado por nombre de usuario. La operación de alta es atómica: `register(username, passwordHash)` usa `putIfAbsent`, de modo que un nombre de usuario sólo puede reclamarse una vez aunque dos peticiones de registro lleguen en paralelo.

```
1 public boolean register(String username, String passwordHash) {  
2     Credential created = new Credential(username, passwordHash);  
3     return byUsername.putIfAbsent(username, created) == null;  
4 }  
5  
6 public Credential find(String username) {  
7     return byUsername.get(username);  
8 }
```

Listing 21: CredentialStore: alta atómica y búsqueda

`register` devuelve `true` si el alta tuvo éxito y `false` si el usuario ya existía (cuando `putIfAbsent` no es `null`); y `find(username)` devuelve la credencial guardada o `null` si no hay ninguna. Al ser un almacén en memoria, las credenciales son locales al proceso y no se replican entre nodos junto con el libro mayor.

5.3. Autenticador y protección de endpoints

`security/Authenticator.java` es la fachada que coordina alta de cuentas, login y autorización de peticiones. Se construye inyectándole un `CredentialStore` (quién existe) y un `Tokens` (prueba de identidad); en `app/Node.java` se crea con `new Authenticator(credentials, tokens)`.

Alta y login. `signup(username, password)` hashlea la contraseña con `Passwords.encode` y delega en `store.register`, devolviendo `false` si el usuario estaba tomado. `login(username, password)` busca la credencial, verifica la contraseña con `Passwords.matches` y, sólo si todo concuerda, emite un JWT con `tokens.issue(username)`; en cualquier fallo (usuario inexistente o contraseña incorrecta) devuelve `null`, sin distinguir el caso para no filtrar qué usuarios existen.

Autorización por Bearer. `authorize(req)` extrae la cabecera `Authorization`, exige el esquema `Bearer` y valida el token:

```
1 public String authorize(Request req) {
2     String header = req.header("Authorization");
3     if (header == null) {
4         return null;
5     }
6     String[] parts = WHITESPACE.split(header.trim(), 2);
7     if (parts.length < 2 || !SCHEME.equalsIgnoreCase(parts[0])) {
8         return null;
9     }
10    String token = parts[1].trim();
11    if (token.isEmpty()) {
12        return null;
13    }
14    try {
15        return tokens.validate(token);
16    } catch (RuntimeException e) {
17        return null;
18    }
19 }
```

Listing 22: `Authenticator.authorize`: validacion del Bearer

El método es tolerante con el formato según el RFC 7235: el esquema se compara con `equalsIgnoreCase` (acepta `Bearer`, `bearer` o `BEARER`) y los espacios sobrantes se absorben partiendo con un patrón de espacios precompilado (`WHITESPACE`). Devuelve el sujeto del token cuando la firma y el emisor son válidos, y `null` en todos los casos de fallo: cabecera ausente, esquema equivocado, token vacío o cualquier `JWTVerificationException` capturada como `RuntimeException`.

Rutas públicas frente a protegidas. El registro de rutas se hace en `app/Node.java`. Son *públicas* `POST /api/register` (atendida por `RegisterEndpoint`) y `POST /api/login` (atendida por `LoginEndpoint`): no requieren token porque su propósito es precisamente obtenerlo. Son *protegidas* las que operan sobre cuentas, como `GET /api/accounts/{id}` (`BalanceEndpoint`) y `POST /api/transactions/transfer` (`TransferEndpoint`). Cada una empieza llamando a `auth.authorize(req)` y, si devuelve `null`, corta con `Reply.status(401)` antes de tocar la lógica de negocio:

```
1 if (auth.authorize(req) == null) {
2     return Reply.status(401);
3 }
```

Listing 23: `BalanceEndpoint`: corte 401 si el token falla

Así, una petición sin cabecera `Authorization`, con un esquema distinto de `Bearer` o con un token mal firmado o de otro emisor recibe un 401 `Unauthorized`, mientras que un token válido deja pasar la petición a la operación correspondiente.

6. API REST y endpoints

Este capítulo documenta el contrato REST del nodo: cada endpoint con su método, ruta, cuerpo, respuesta y códigos de estado. Toda la API se sirve sobre el mismo router (`net/Routes.java`) y dos de los endpoints exigen un JWT, siguiendo la estructura REST + JWT que pide la rúbrica.

6.1. Mapa de rutas

El enrutador vive en `net/Routes.java`. Es un *router* por método y ruta: cada patrón se parte en segmentos separados por `/` y un segmento escrito como `{nombre}` es una variable de un solo segmento; el resto debe coincidir por igualdad exacta de cadena (no es coincidencia por

prefijo más largo). El método `Routes.match()` resuelve un método y una ruta concreta contra las rutas registradas con una semántica clara de tres resultados: si el método y la forma de los segmentos coinciden devuelve el `Endpoint` con estado 200; si algún patrón coincide con la ruta pero ningún método coincide devuelve 405 (*method not allowed*); y si ningún patrón coincide con la ruta devuelve 404.

El cableado real de rutas a *endpoints* ocurre en `app/Node.java`, mediante llamadas a `routes.register(...)`. La siguiente tabla resume cada endpoint registrado, con su método, ruta, si exige autenticación con *token* y su propósito.

Método	Ruta	Auth	Propósito
POST	/api/register	No	Crea una credencial de usuario.
POST	/api/login	No	Verifica credencial y emite un JWT.
GET	/api/accounts/{id}	Sí	Devuelve el saldo de una cuenta.
POST	/api/transactions/transfer	Sí	Transfiere dinero entre cuentas.
GET	/api/stats	No	Métricas de <i>este</i> nodo.
GET	/panel	No	Métricas agregadas del clúster.
GET	/	No	Sirve el tablero de una sola página.

Solo los dos endpoints de negocio (`/api/accounts/{id}` y `/api/transactions/transfer`) exigen un *token* válido; los de autenticación y los de monitoreo son públicos. La columna *Auth* se valida con `Authenticator.authorize()` dentro de cada endpoint, no en el router.

6.2. Autenticacion: register y login

Registro de usuario

`api/RegisterEndpoint.java` atiende `POST /api/register` y crea una credencial nueva. Acepta un cuerpo JSON con la forma `{"username": "...", "password": "..."}` , lo parsea con `Gson` y delega la persistencia en `Authenticator.signup()`, que cifra la contraseña y la guarda en el `CredentialStore`. El endpoint primero verifica el método (responde 405 si no es `POST`); si el JSON está mal formado o falta `username/password` responde 400 con `{"error": "bad_request"}`; si el usuario aún no existía devuelve 201 con `{"status": "created"}`; y si el nombre ya estaba tomado devuelve 409 con `{"error": "username_taken"}`. La distinción entre creado (201) y duplicado (409) la decide directamente el valor booleano que regresa `auth.signup(username, password)`.

Inicio de sesión

`api/LoginEndpoint.java` atiende `POST /api/login`: verifica la credencial y, en caso de éxito, emite un JWT. Recibe el mismo cuerpo `{"username": "...", "password": "..."}` y delega en `Authenticator.login()`, que busca la credencial en el `CredentialStore`, compara la contraseña con `Passwords.matches()` y solo si coinciden mina un *token* firmado con `Tokens.issue()`. Es decir, el login valida credenciales reales: si el usuario no existe o la contraseña no coincide, `login()` regresa `null` y el endpoint responde 401 con `{"error": "invalid_credentials"}`. Con credenciales válidas responde 200 con `{"token": "<jwt>"}`; un JSON mal formado produce 400 y un método distinto de `POST` produce 405. El cliente reutiliza ese *token* en la cabecera `Authorization: Bearer <jwt>` para las operaciones protegidas.

```

1 POST /api/login
2 Content-Type: application/json
3
4 {"username": "juan", "password": "secreto"}
5
6 200 OK
7 {"token": "<jwt>"}
```

```

8
9 401 Unauthorized
10 {"error": "invalid_credentials"}

```

Listing 24: Contrato JSON de POST /api/login (petición y respuesta)

6.3. Operaciones de negocio: balance y transfer

Consulta de saldo

`api/BalanceEndpoint.java` atiende GET /api/accounts/{id} y devuelve la instantánea de una cuenta. Exige un JWT válido: lo primero que hace, tras descartar métodos distintos de GET (405), es llamar a `auth.authorize(req)`; si regresa null responde 401. Luego resuelve la variable de ruta `id` con `req.pathParam("id")`: si no es un entero (no encaja en el patrón `-?\d+`) responde 400 con `{"error": "bad_id"}`; si es numérico pero se sale del rango de `int`, o si la cuenta no está registrada, responde 404. Con una cuenta válida construye el cuerpo con la forma obligatoria del contrato, donde las llaves son exactamente `id`, `propietario` y `balance`, por ejemplo `{"id":125,"propietario":"JUAN MOLINAR HERNANDEZ","balance":15750.25}`. Un detalle clave: `id` y `balance` deben serializarse como *números* JSON (sin comillas); como `Money.toDecimal()` devuelve un `String`, el saldo se envuelve en un `BigDecimal` para que `Gson` lo emita como número y no como cadena entre comillas.

Transferencia

`api/TransferEndpoint.java` atiende POST /api/transactions/transfer y mueve dinero entre dos cuentas. También exige JWT (401 sin *token*) y rechaza métodos distintos de POST (405). El cuerpo obligatorio usa la forma `{"sourceAccountId":"123","targetAccountId":"456","amount":200.00}` donde los identificadores son *cadena*s y `amount` es decimal. El monto decimal se convierte a centavos con `Money.toCents()` y la operación se orquesta en `Bank.transfer()`, que valida la solicitud, aplica el movimiento atómico, estampa un número de secuencia fresco y preserva la invariante de conservación. Un éxito devuelve 200 con `{"status":"ok","seq":<seq>}` llevando la secuencia asignada. Un rechazo se manifiesta como una `TransferException`: su `code()` determina el estado, donde `"no_such_account"` mapea a 404 y el resto de códigos (`"self_transfer"`, `"bad_amount"`, `"low_balance"`) mapean a 400, y el cuerpo refleja `{"error":<code>}`. Un JSON mal formado, un identificador no numérico o un monto inválido producen 400 con `{"error": "bad_request"}`.

```

1 POST /api/transactions/transfer
2 Authorization: Bearer <jwt>
3 Content-Type: application/json
4
5 {"sourceAccountId":"123","targetAccountId":"456","amount":200.00}
6
7 200 OK
8 {"status":"ok","seq":42}
9
10 400 Bad Request
11 {"error":"low_balance"}
12
13 404 Not Found
14 {"error":"no_such_account"}

```

Listing 25: Contrato JSON de POST /api/transactions/transfer

El mapeo de `TransferException` a estado HTTP se concentra en pocas líneas, copiadas tal cual del endpoint:

```

1 } catch (TransferException e) {
2     int status = "no_such_account".equals(e.code()) ? 404 : 400;

```

```

3   JsonObject err = new JsonObject();
4   err.addProperty("error", e.code());
5   return Reply.json(status, gson.toJson(err));
6 }

```

Listing 26: Mapeo de rechazos de transferencia a estado REST en TransferEndpoint.java

6.4. Endpoints de monitoreo

Tres endpoints alimentan el tablero. Aquí se resumen; su funcionamiento detallado se trata en el capítulo 9.

`api/StatsEndpoint.java` atiende GET `/api/stats` y reporta las métricas de *este* nodo. Serializa la instantánea viva de `NodeStats` llamando a `stats.toJson()`, que produce un objeto JSON con las llaves `nodeId`, `role`, `status`, `accountCount`, `totalBalance`, `transferCount`, `lastTxId`, `cpuPercent`, `ramPercent`, `diskPercent` y `gcsCount` (el número de transacciones almacenadas en Cloud Storage). Responde 405 ante un método distinto de GET.

`api/PanelEndpoint.java` atiende GET `/panel` y agrega las métricas de todo el clúster. Combina la instantánea local (`stats.toJson()`) con las métricas que obtiene por HTTP del endpoint `/api/stats` de cada par listado en `TES_PEERS` (vía `NodeConfig`), y produce un único documento JSON con la forma `{"nodes":[...]}`, una entrada por nodo, para que el tablero dibuje el clúster en una sola página. Cada par se consulta de forma concurrente con un `HttpClient` de *timeout* acotado; un par inalcanzable o lento no tumba la respuesta: se reporta como nodo caído (`"status":"down"`) en lugar de fallar todo el panel. Además recuerda el último `nodeId` que cada par reportó mientras estuvo arriba, de modo que la tarjeta de un nodo caído conserve su identidad real durante la demostración de tolerancia a fallos.

`api/DashboardEndpoint.java` atiende GET `/` y sirve el tablero de una sola página: transmite el recurso `dashboard.html` desde el *classpath* como respuesta HTML. La propia página sondea `/panel` (y ofrece un control de *Refresh*) para mostrar el estado de cada nodo, su número de cuentas, el saldo total, el número de transferencias, el id de la última transacción, %CPU, %RAM, %Disco y el conteo de transacciones en Cloud Storage. Responde 405 ante un método distinto de GET y 404 si el recurso HTML no existe.

7. Replicacion y reanudacion exacta por secuencia

Este capítulo es el punto clave de la rubrica de tolerancia a fallos: cuando se mata el *proceso* de una replica (no la maquina virtual) y luego revive, la replica debe reanudar **desde el numero de secuencia exacto** en el que se quedo, pidiendo al lider solo el delta faltante. El anti-patron de “borrar el estado y volver a descargar todo” funciona pero no escala y cuesta puntos. Las piezas que lo implementan son `cluster/ReplicaFeed.java` (lado lider), `cluster/ReplicaSync.java` (lado replica), `cluster/WireCodec.java` (formato de mensaje) y `durable/ReplicaCheckpoint.java` (durabilidad que sobrevive a la muerte del proceso).

7.1. Protocolo de replicacion

El lider publica su flujo de *commits* sobre TCP plano a traves de `cluster/ReplicaFeed.java`, que escucha en el puerto de replicacion `TES_REPL_PORT`. Cuando una replica se conecta, primero anuncia hasta donde ya esta al dia con un saludo de una linea `CATCHUP <seq>`; el lider lo lee en `serveReplica()` y `parseCatchup()`. La respuesta consta de dos fases: primero reproduce, en orden, todas las transferencias del `core/TransferLog.java` cuya secuencia es estrictamente mayor que esa marca (`log.since(since)`), y despues sigue transmitiendo en vivo cada nuevo commit.

El detalle decisivo es que **el lider no espera confirmacion de las replicas**: nunca bloquea su throughput por una replica lenta o caída. `ReplicaFeed` es un `core/CommitListener.java`

invocado en linea sobre los hilos de commit del lider, asi que no puede bloquearse ahi. Cada suscriptor tiene su propia cola acotada (`QUEUE_CAPACITY = 8192`) y un hilo escritor dedicado; `onCommit()` solo hace un **offer** no bloqueante en cada cola. Una replica cuyo socket se atasca solo llena su *propia* cola; al desbordarse, se descarta ese suscriptor y se cierra su socket, sin afectar la ruta de commit ni a las demas replicas.

```

1 @Override
2 public void onCommit(Transfer t) {
3     String line = WireCodec.encode(t);
4     for (Subscriber subscriber : subscribers) {
5         if (!subscriber.enqueue(line)) {
6             drop(subscriber);
7         }
8     }
9 }

```

Listing 27: ReplicaFeed.onCommit: fan-out no bloqueante hacia las replicas

El formato de cada mensaje lo define `cluster/WireCodec.java`: una transferencia se serializa como un unico objeto JSON por linea con los cuatro campos que un par necesita para reaplicar el commit (la secuencia, las cuentas origen y destino, y el monto en centavos). Al ser texto y un objeto por linea, la replica puede leer el flujo con un lector de lineas ordinario. `WireCodec` es un ayudante sin estado (constructor privado) que solo expone `encode()` y `decode()`; los campos numericos se leen con su ancho nativo (`getAsLong/getAsInt`) para que `seq` y `cents` de 64 bits sobrevivan el viaje de ida y vuelta sin perder precision.

```

1 public static String encode(Transfer t) {
2     JsonObject object = new JsonObject();
3     object.addProperty("seq", t.seq());
4     object.addProperty("from", t.from());
5     object.addProperty("to", t.to());
6     object.addProperty("cents", t.cents());
7     return object.toString();
8 }
9
10 public static Transfer decode(String line) {
11     try {
12         JsonObject object = JsonParser.parseString(line).getAsJsonObject();
13         long seq = object.get("seq").getAsLong();
14         int from = object.get("from").getAsInt();
15         int to = object.get("to").getAsInt();
16         long cents = object.get("cents").getAsLong();
17         return new Transfer(seq, from, to, cents);
18     } catch (RuntimeException ex) {
19         throw new IllegalArgumentException("malformed transfer line: " + line,
20             ex);
21     }
22 }

```

Listing 28: WireCodec: codificar y decodificar una transferencia como linea JSON

La linea de cable es entonces `{"seq":...,"from":...,"to":...,"cents":...}` seguida de un salto de linea. El orden se mantiene sin un candado global de difusion: el lider agrega la transferencia al log *antes* de notificar a los oyentes, y un suscriptor se registra en la audiencia en vivo *antes* de que su escritor lea el backlog, de modo que todo commit llega a la replica por el backlog, por la cola en vivo, o (inofensivamente) por ambos. Los duplicados y el ligero reordenamiento en esa frontera los absorbe la replica.

7.2. La replica: aplicar y seguir el flujo

Del lado seguidor, `cluster/ReplicaSync.java` mantiene al nodo alineado con el lider. Marca al puerto de replicacion del `TES_LEADER_HOST` y abre la conversacion con el saludo `CATCHUP <seq>`, donde la secuencia es `Bank.lastSeq()`, la transferencia mas alta que el `core/Bank.java` local ya aplico. El lider entonces transmite, como lineas JSON, primero el backlog posterior a esa marca y luego cada commit en vivo; cada linea recibida se vuelve a convertir en un `core/Transfer.java` con `WireCodec.decode()`.

Como el lider reparte los commits desde hilos concurrentes, una secuencia mas alta puede llegar antes que una mas baja. Por eso `ReplicaSync` aplica las transferencias a traves de un pequeno *buffer de reordenamiento* (`TreeMap<Long, Transfer> pending`) que garantiza que el banco siempre las vea en orden de secuencia estricto y sin huecos. `bufferAndDrain()` guarda las llegadas fuera de orden hasta que las secuencias faltantes se completan, y entonces las vacia de forma contigua hacia `Bank.applyReplicated()`. Una transferencia con secuencia igual o menor a la marca del banco es un duplicado y se descarta, asi que un solapamiento reenviado tras una reconexion es inofensivo.

```

1 private void bufferAndDrain(TreeMap<Long, Transfer> pending, Transfer t) {
2     if (t.seq() > bank.lastSeq()) {
3         pending.put(t.seq(), t);
4     }
5     while (!pending.isEmpty() && pending.firstKey() <= bank.lastSeq() + 1) {
6         Transfer next = pending.pollFirstEntry().getValue();
7         if (next.seq() > bank.lastSeq()) {
8             bank.applyReplicated(next);
9         }
10    }
11 }

```

Listing 29: `ReplicaSync.bufferAndDrain`: reordenar y aplicar en secuencia contigua

El enlace se trata como poco confiable. El lector corre dentro de `syncLoop()`, un bucle que, ante cualquier caida, espera brevemente (`RECONNECT_DELAY_MS = 1000`) y vuelve a marcar, reemitiendo `CATCHUP` con la marca mas reciente del banco para que la reproduccion se reanude desde ahi y no desde cero. Cada reconexion estrena un buffer `pending` nuevo, de modo que cualquier resto fuera de orden del enlace anterior se descarta. Una linea corrupta tampoco mata al lector: rompe el ciclo interno para que el bucle externo reconecte y el lider reenvie desde la marca actual.

7.3. Checkpoint local y reanudacion exacta

Todo lo anterior basta para sobrevivir a una caida de *red*: la marca vive en memoria en `Bank.lastSeq()`. Pero si matan el *proceso* de la replica, esa memoria se pierde. Aqui entra `durable/ReplicaCheckpoint.java`, que persiste en almacenamiento durable lo necesario para reanudar exacto: la marca de secuencia y los saldos. Un checkpoint es un unico objeto de Cloud Storage, sobrescrito en el mismo lugar, con la forma `{v, watermark, idBase, count, balances}`. El `watermark` es la secuencia mas alta aplicada; `balances` es una instantanea posicional donde el casillero `k` corresponde al saldo de la cuenta `idBase + k`, asi que la posicion codifica el id y no se guarda una clave por cuenta. La marca y los saldos se capturan juntos bajo el monitor del banco (`Bank.snapshotInto()`), de modo que el par es consistente aunque el hilo de sincronizacion este aplicando transferencias.

Al revivir, `load()` se ejecuta *antes* de que arranque la replicacion: restaura los saldos sobre el ledger ya cargado del CSV y siembra la marca del banco con `Bank.seedLastSeq(watermark)`, que sube (nunca baja) el `lastSeq` y el contador de secuencia. Asi, el `CATCHUP` posterior lleva la marca restaurada `X` en vez de 0, y el lider solo transmite el delta, es decir desde `X+1` (`log.since(X)`).

devuelve las transferencias *estrictamente mayores* que X). Un checkpoint ausente, ilegible o incompatible (version, `idBase` o numero de cuentas que no coinciden) se ignora y el arranque cae a un catch-up completo; `load()` nunca lanza excepcion.

El mensaje que evidencia la reanudacion exacta – el literal “Me quede en la secuencia X, enviame desde X+1” que pide la rubrica – lo imprime `cluster/ReplicaSync.java` justo antes de cada CATCHUP, en el camino de revival de toda replica (con o sin checkpoint). Captura la marca una sola vez para que el mensaje y el CATCHUP coincidan:

```
1 long seq = bank.lastSeq();
2 System.out.println("[replica] Me quede en la secuencia " + seq
3     + ", enviame desde " + (seq + 1));
4 out.write(("CATCHUP " + seq + "\n").getBytes(StandardCharsets.UTF_8));
```

Listing 30: ReplicaSync: imprime el punto de reanudacion antes del CATCHUP

Cuando ademas hay checkpoint, `ReplicaCheckpoint.load()` deja constancia de la marca restaurada y `app/Node.java` la confirma, de modo que la salida tipica al revivir es:

```
1 [checkpoint] restored checkpoint/nodo-2.json at watermark 4127
2 [node] replica resuming at sequence 4127
3 [replica] Me quede en la secuencia 4127, enviame desde 4128
```

Listing 31: Salida tipica al revivir el proceso de una replica

Con la marca 4127 sembrada, `ReplicaSync` envia CATCHUP 4127 y el lider reproduce desde la 4128 en adelante: esa es la reanudacion exacta por secuencia. La cadencia de guardado tiene dos partes: un paro ordenado (el *shutdown hook* en `Node.java`) escribe el estado exacto de la ultima transferencia aplicada – y la demostracion en vivo apaga la VM, que es un paro ordenado –, mientras que un demonio periodico (`ReplicaCheckpoint.start()`) escribe un piso para que un *crash* duro pierda a lo sumo un intervalo. Un `save()` cuya marca no cambio desde el ultimo se omite (`lastSavedWatermark`), asi que una replica ociosa no cuesta nada.

7.4. Por que no “borra y descarga”

El contraste con el anti-patron es el corazon de la rubrica. “Borrar el estado y bajar todo de nuevo” significa que la replica, al revivir, descarta sus saldos y pide al lider la historia completa desde la secuencia 0. Eso funciona, pero el costo de catch-up crece con *toda* la historia del banco, no con lo que la replica perdio mientras estuvo caida: no escala y, segun la rubrica, baja la calificacion de 25 a 13 puntos.

La implementacion aqui reanuda por *delta exacto*. Gracias al checkpoint durable, la replica conoce su `watermark` al arrancar y solo pide CATCHUP `<watermark>`; el lider responde con `log.since(watermark)`, que son unicamente las transferencias `watermark+1`, `watermark+2`, ... que ocurrieron durante la caida. El trabajo de recuperacion es proporcional a la *brecha*, no al tamano total del estado. Y como `applyReplicated()` descarta cualquier secuencia ya aplicada y el buffer de reordenamiento garantiza orden contiguo, reanudar desde la marca es seguro aunque el lider reenvie un solapamiento: el resultado converge al estado del lider sin reprocesar la historia entera.

8. Persistencia durable en Cloud Storage

La durabilidad del mini-banco se apoya en un *log* de transferencias que vive en un bucket de Google Cloud Storage. Cada transferencia confirmada por el lider termina escrita en esa bitacora, de modo que si caen los tres nodos a la vez el lider pueda renacer y reconstruir todos los saldos releendo el log. Tres clases cooperan: `durable/Journal.java` orquesta la escritura y la recuperacion, `durable/GcsStore.java` habla con la JSON API de Cloud Storage y

`durable/GcsAuth.java` obtiene los tokens de acceso. La unidad que se persiste es siempre un `core/Transfer.java`, el mismo registro que viaja por el cluster.

8.1. La bitacora de transacciones

El `durable/Journal.java` es un orquestador de durabilidad que solo existe en el lider (las replicas nunca poseen un `Journal`). Implementa la interfaz `core/CommitListener.java`: una vez registrado sobre el banco del lider, su metodo `onCommit(Transfer)` recibe cada transferencia recién confirmada. Lo importante es que *no* sube nada a GCS en ese momento: solo encola la transferencia y regresa de inmediato. Así, la respuesta HTTP de una transferencia nunca se bloquea esperando un viaje de ida y vuelta a Cloud Storage.

```
1 @Override
2 public void onCommit(Transfer transfer) {
3     queue.offer(transfer);
4 }
```

Listing 32: `onCommit` solo encola; la subida ocurre fuera del hot path

La cola es un `BlockingQueue` que un unico hilo de fondo (`journal-writer`, marcado como *daemon*) consume en `drainLoop()`. Ese hilo espera con `poll(POLL_MS, ...)` a que aparezca una transferencia, y cuando la toma vacia de golpe todo lo que haya disponible en la cola hasta un maximo de `MAX_BATCH` (1000) con `drainTo`, componiendo un solo lote. El lote completo se sube como un unico objeto mediante `store.putBatch(batch)` y, solo si la subida tuvo exito, se avanza el contador `stored` en el tamaño del lote. El agrupado por lotes es lo que permite que el log durable siga el ritmo de una tasa alta de commits sin un objeto por transferencia.

```
1 batch.clear();
2 batch.add(first);
3 queue.drainTo(batch, MAX_BATCH - 1);
4 try {
5     store.putBatch(batch);
6     stored.addAndGet(batch.size());
7 } catch (IOException e) {
8     System.err.println("journal: could not persist batch of " + batch.size()
9         + " ending at seq " + batch.get(batch.size() - 1).seq()
10        + ": " + e.getMessage());
11 }
```

Listing 33: `drainLoop` compone un lote y lo sube como un solo objeto

Cada entrada del log es exactamente un `core/Transfer.java`: un registro inmutable con un numero de secuencia monotono asignado por el ledger al confirmar (`seq`), la cuenta origen (`from`), la cuenta destino (`to`) y el monto movido en centavos (`cents`). Es la misma forma que serializan el codec del cluster y el journal de GCS.

```
1 public record Transfer(long seq, int from, int to, long cents) {
2 }
```

Listing 34: La unidad persistida: el registro `Transfer` (`core/Transfer.java`)

El contador `stored()` reporta cuantas transferencias estan *realmente* subidas a GCS; queda por debajo del numero de commits aplicados en exactamente la profundidad de la cola, que es la cifra durable que debe mostrar el dashboard. Un fallo de subida se registra en `stderr` pero no detiene el hilo: el lazo corre mientras `running` sea verdadero o quede algo en la cola. Un lote cuya subida falla se descarta y deja un hueco en el log durable; para que ese hueco no pase inadvertido, `recover()` valida la contiguidad al arrancar (el log es contiguo desde 1) y avisa por `stderr` si una secuencia no es la sucesora esperada, en vez de recuperar en silencio un log incompleto.

Esta estrategia asincrona tiene una ventana de durabilidad explicita. En un apagado ordenado, `stop()` pone `running` en falso y hace `join` al escritor para drenar la cola, de modo que

toda transferencia ya confirmada quede subida antes de que el proceso termine; `app/Node.java` registra ese `stop()` como *shutdown hook*. En cambio, una muerte abrupta del proceso deja la cola en vuelo como la ventana de durabilidad inherente a un journaling asincrono: como el commit responde al cliente antes de subir a GCS, una transferencia confirmada puede perderse en un *kill* duro. Ese es el precio consciente de mantener Cloud Storage fuera del camino critico.

8.2. Acceso a Cloud Storage

El `durable/GcsStore.java` es un cliente delgado sobre la JSON API REST de Cloud Storage, construido directamente con `java.net.http.HttpClient` y sin el SDK de Google. Todos los objetos del journal cuelgan del prefijo `journal/`. El metodo `put(Transfer)` existe y escribiria una transferencia suelta como `journal/tx-seq.json`, pero el camino vivo no lo usa: `Journal` siempre llama a `putBatch`, que compone muchas transferencias en un solo objeto `journal/batch-min-max.json` (un arreglo JSON), nombrado por la secuencia minima y maxima del lote. Cloud Storage no tiene *append*; componer objetos es la alternativa sancionada por el contrato.

Las subidas usan el endpoint de *simple media upload* (`UPLOAD_BASE`) con `uploadType=media`, mientras que listar y leer usan la JSON API (`API_BASE`). Toda peticion lleva la cabecera `Authorization: Bearer` con un token de `GcsAuth` y se serializa con `gson`. Ademas, `GcsStore` ofrece `putCheckpoint/getCheckpoint` para que una replica guarde su snapshot bajo el prefijo `checkpoint/`; estos quedan deliberadamente fuera de `journal/` para que el `readAll()` del lider nunca los liste ni intente parsearlos como `Transfer`.

```

1 long min = batch.stream().mapToLong(Transfer::seq).min().getAsLong();
2 long max = batch.stream().mapToLong(Transfer::seq).max().getAsLong();
3 String name = PREFIX + "batch-" + min + "-" + max + ".json";
4 String url = UPLOAD_BASE + bucket + "/o?uploadType=media&name=" + enc(name);
5 HttpRequest request = HttpRequest.newBuilder(URI.create(url))
6     .timeout(HTTP_TIMEOUT)
7     .header("Authorization", "Bearer " + auth.accessToken())
8     .header("Content-Type", "application/json")
9     .POST(HttpRequest.BodyPublishers.ofString(gson.toJson(batch)))
10    .build();

```

Listing 35: `putBatch` compone el lote en un solo objeto `journal/batch-min-max.json`

Toda peticion pasa por `sendWithRetry`, que reintenta hasta `MAX_ATTEMPTS` (3) veces ante un fallo de red transitorio (`IOException`: una conexion caida, un timeout) y, si todos los intentos fallan, relanza la excepcion. Esto hace que la recuperacion en frio aborta de forma ruidosa en lugar de servir silenciosamente un estado solo-base. Un estado HTTP no-2xx no se reintenta: se devuelve al llamador, que decide como tratarlo (por ejemplo, `getCheckpoint` interpreta un 404 como “aun no existe” y devuelve `null`).

8.2.1. Autenticacion con la cuenta de servicio

El `durable/GcsAuth.java` acuna tokens de acceso OAuth2 para la JSON API usando el flujo de *key-file* de cuenta de servicio, tambien sin SDK de Google (el metadata server de la VM no se usa a proposito). En el constructor lee el JSON de la cuenta de servicio apuntado por `TES_GCS_KEYFILE` y extrae `client_email`, `token_uri` (con un endpoint por defecto si falta) y `private_key`, del que reconstruye la llave RSA PKCS8.

Para obtener un token, `buildAssertion()` arma un JWT con cabecera RS256 y un claim set cargado con `iss`, `scope` (`devstorage.read_write`), `aud`, `iat` y `exp`, lo firma con `SHA256withRSA` sobre la llave privada y lo codifica en Base64 URL sin relleno. No se envia `sub` (eso es solo para delegacion de dominio y aqui seria rechazado como `unauthorized_client`).

```

1 JsonObject claims = new JsonObject();

```

```

2 claims.addProperty("iss", clientEmail);
3 claims.addProperty("scope", SCOPE);
4 claims.addProperty("aud", tokenUri);
5 claims.addProperty("iat", now);
6 claims.addProperty("exp", now + TOKEN_TTL_SECONDS);

```

Listing 36: Claims del JWT que GcsAuth autofirma (RS256)

El JWT firmado se intercambia en `refresh()` por un POST `x-www-form-urlencoded` al `token_uri` con el grant `urn:ietf:params:oauth:grant-type:jwt-bearer`, y del cuerpo se lee `access_token`. `accessToken()` cachea el token y lo reusa hasta poco antes de expirar (margen `REFRESH_SKEW_SECONDS` de 60 s), acunando uno nuevo solo cuando hace falta; el metodo es `synchronized` porque el hilo escritor lo consulta en cada subida.

El alta de la cuenta de servicio se hace una sola vez, segun describe `SETUP-GCS.md`: crear la cuenta, otorgarle `roles/storage.objectAdmin` sobre el bucket (no basta `objectCreator`, porque el log no solo sube sino que *relee* en el arranque en frio) y descargar la llave `credentials.json`. Esa llave nunca se sube al repositorio; en la VM se copia aparte y se apunta `TES_GCS_KEYFILE` a su ruta, mientras `TES_BUCKET` lleva el nombre del bucket (solo en el nodo lider).

8.3. Recuperacion ante desastre total

Cuando el lider arranca, `app/Node.java` corre la recuperacion durable antes de aceptar trafico, siempre que Cloud Storage este configurado (si faltan `TES_BUCKET` o `TES_GCS_KEYFILE`, el lider sigue sirviendo todos los endpoints en memoria y el journal simplemente no se activa). El arranque crea el `GcsAuth` y el `GcsStore`, instancia el `Journal` y llama a `recover`, pasandole `bank::applyReplicated` como la accion que aplica cada transferencia recuperada al ledger.

```

1 GcsAuth auth = new GcsAuth(Path.of(config.gcsKeyfile()));
2 GcsStore store = new GcsStore(auth, config.bucket());
3 journal = new Journal(store);
4 int recovered = journal.recover(bank::applyReplicated);
5 journal.start(recovered);
6 bank.addCommitListener(journal);

```

Listing 37: Arranque del lider: recuperar, sembrar el contador y registrar el journal (`app/Node.java`)

Dentro de `recover(CommitListener apply)`, el journal pide a `store.readAll()` todas las transferencias persistidas y reaplica cada una en orden al ledger. El metodo `readAll()` de `GcsStore` lista los objetos bajo el prefijo `journal/` (siguiendo la paginacion por `nextPageToken` de la JSON API), baja cada uno y los aplanas: si el objeto es un arreglo (un lote) expande cada elemento, y si es un objeto suelto lo convierte directo. Como cada `Transfer` carga su `seq`, el “donde quedo” no depende del nombre de los objetos ni de un cursor persistido del lider: `readAll()` reordena todo por secuencia al final, asi que el ledger se reconstruye exactamente hasta la ultima secuencia confirmada sin huecos ni duplicados de orden.

```

1 JsonElement parsed = JsonParser.parseString(response.body());
2 if (parsed.isJsonArray()) {
3     for (JsonElement element : parsed.getAsJsonArray()) {
4         transfers.add(gson.fromJson(element, Transfer.class));
5     }
6 } else {
7     transfers.add(gson.fromJson(parsed, Transfer.class));
8 }
9 // ...
10 transfers.sort(Comparator.comparingLong(Transfer::seq));

```

Listing 38: `readAll` aplanas lotes y transferencias sueltas, y reordena por secuencia (`GcsStore`)

Hecha la reubicación, **recover** devuelve cuantas transferencias se recuperaron y **start(recovered)** siembra con ese valor el contador **stored** (para que el dashboard arranque mostrando lo ya durable) y lanza el hilo escritor que sube los commits vivos a partir de ese punto. El líder queda entonces sirviendo con los saldos completos reconstruidos desde el log, que el dashboard muestra como saldos recuperados. La garantía es fuerte para todo lo que alcanzo a subirse a GCS y se vio reforzada por el *shutdown hook* que drena la cola en un apagado ordenado; lo único no recuperable es la ventana asincrónica en vuelo descrita antes, ante una caída abrupta.

9. Dashboard de observabilidad

El dashboard es la cara visible del cluster: una sola página web que monitorea los tres nodos a la vez y, para cada uno, pinta los datos de negocio (cuentas, saldo, transferencias, secuencia de la última transacción) junto con las métricas de infraestructura del sistema operativo de su máquina (CPU, RAM y Disco). El armado descansa en tres endpoints HTTP, registrados en `app/Node.java` sobre el router `net/Routes.java`: `GET /` sirve la página, `GET /panel` agrega la vista de todo el cluster y `GET /api/stats` expone las métricas de un solo nodo. El extra de la rúbrica (auto-refresh, sin tener que pulsar “Refrescar”) se cubre con un `setInterval` de polling en el JavaScript de la página.

9.1. La interfaz web

La página vive como recurso estático en `src/main/resources/dashboard.html` y se entrega tal cual desde el classpath por `api/DashboardEndpoint.java`: ese endpoint solo valida que el método sea `GET` (responde 405 si no) y vuelca el HTML con `Reply.html(200, ...)`, leyendo el recurso `/dashboard.html` en cada petición. No tiene colaboradores ni lógica de negocio; toda la inteligencia está en el JavaScript del propio documento.

Es una SPA en HTML/CSS/JS puro, sin dependencias ni librerías externas. Al cargar, y cada vez que se refresca, la función `loadPanel()` hace `fetch("/panel")` pidiendo JSON y, con la respuesta, repinta tres zonas: el *hero* (saldo total del cluster, número de cuentas y última transacción, tomados del nodo líder), un sello de consistencia que compara los saldos totales de los nodos en línea para indicar si el invariante se conserva (cada total proviene del snapshot atómico de `totalCents()`, así que el sello no parpadea por lecturas rotas bajo carga), y una tarjeta por nodo. Cada tarjeta dibuja el rol (*Líder/Replica*), el estado (*Activo/Caído*), el saldo del nodo, los cuatro contadores de negocio (cuentas, transferencias, última tx y transacciones en Cloud Storage) y tres barras (*gauges*) para CPU, RAM y Disco. Si `/panel` falla, se muestra un *banner* de error en vez de romper la página.

El auto-refresh, que da los 15 puntos extra, es un *toggle* “En vivo” que **arranca encendido por defecto**: el checkbox lleva `checked` y, al cargar, la página llama `toggleReactive(checkbox.checked)`, de modo que el tablero empieza a refrescarse solo *sin pulsar “Refrescar”*, que es justo lo que pide el extra. `toggleReactive()` arranca un `setInterval` que vuelve a llamar a `loadPanel()` cada `REACTIVE_INTERVAL_MS` (1500 ms); al desactivar el toggle se limpia el temporizador con `clearInterval`.

```

1 // Poll cadence (ms) used when the reactive toggle is enabled.
2 const REACTIVE_INTERVAL_MS = 1500;
3 let timer = null;
4 // ...
5 function toggleReactive(enabled) {
6   if (timer) { clearInterval(timer); timer = null; }
7   if (enabled) timer = setInterval(loadPanel, REACTIVE_INTERVAL_MS);
8   loadPanel();
9 }

```

Listing 39: Polling automático del dashboard (dashboard.html)

9.2. Datos de negocio

Los datos de negocio de cada nodo nacen en `app/NodeStats.java`, un *snapshot* en vivo que reparte las cifras entre dos colaboradores: del `core/Ledger.java` salen el número de cuentas (`accountCount()` vía `ledger.size()`) y el saldo total en centavos (`totalCents()`); del `core/Bank.java` salen el número de transferencias confirmadas (`transferCount()` vía `bank.appliedCount()`) y la secuencia de la última transacción aplicada (`lastTxId()` vía `bank.lastSeq()`). El método `toJson()` serializa todo a un `JsonObject` cuyas claves coinciden exactamente con los campos que consume el JavaScript: `nodeId`, `role`, `status`, `accountCount`, `totalBalance` (convertido a decimal con `Money.toDecimal()`), `transferCount`, `lastTxId`, los tres porcentajes y `gcsCount`.

Quien expone ese JSON de un solo nodo es `api/StatsEndpoint.java` en la ruta `GET /api/stats`: valida el método y devuelve `Reply.json(200, stats.toJson())`. Sobre él se monta `api/PanelEndpoint.java` (ruta `GET /panel`), que arma la vista del cluster completo: parte del `toJson()` del nodo local y, en paralelo, consulta el `/api/stats` de cada peer listado en `TES_PEERS` (`NodeConfig`). Las peticiones se lanzan concurrentes con `CompletableFuture` para que un nodo caído sume a lo sumo un *timeout* a la latencia del panel, no uno por peer; un peer inalcanzable no rompe la respuesta, sino que se reporta como nodo en estado `down`. El resultado es un único documento `{"nodes": [...]}` con una entrada por nodo.

```

1  StringBuilder nodes = new StringBuilder();
2  nodes.append("{\"nodes\":[\"").append(stats.toJson());
3  for (CompletableFuture<String> peer : pending) {
4      nodes.append(',').append(peer.join());
5  }
6  nodes.append("]}\");
7  return Reply.json(200, nodes.toString());

```

Listing 40: PanelEndpoint agrega el nodo local y los peers

9.3. Métricas de infraestructura del SO

Las tres barras de CPU, RAM y Disco no son inventadas: `NodeStats` las lee del sistema operativo de cada VM con Java puro, sin librerías externas. El CPU se toma de `/proc/stat`: `cpuPercent()` suma los *jiffies* de la línea agregada `cpu` y calcula la fracción ocupada como $1 - \Delta \text{idle} / \Delta \text{total}$ entre dos lecturas consecutivas (entre dos refrescos del dashboard), guardando el muestreo previo bajo `cpuLock` para que las peticiones concurrentes no se pisen. La RAM viene de `/proc/meminfo` (`ramPercent()` compara `MemTotal` contra `MemAvailable`). El Disco se calcula con `java.io.File` sobre la raíz `/`: `diskPercent()` usa `getTotalSpace()` y `getUsableSpace()`. Las tres pasan por `clamp()` al rango `[0, 100]` y por `round1()` a un decimal antes de serializarse.

```

1  public double ramPercent() {
2      try {
3          long total = readMeminfo("MemTotal");
4          long available = readMeminfo("MemAvailable");
5          if (total <= 0L) {
6              return 0.0;
7          }
8          return clamp((double) (total - available) / total * 100.0);
9      } catch (RuntimeException | IOException e) {
10         return 0.0;
11     }
12 }

```

Listing 41: Lectura de %RAM y %Disco del SO (NodeStats)

Estos porcentajes viajan al tablero por la misma vía que los datos de negocio: `toJson()` los empaqueta como `cpuPercent`, `ramPercent` y `diskPercent`; `StatsEndpoint` los publica en `/api/stats`, `PanelEndpoint` los agrega en `/panel`, y el JavaScript de `dashboard.html` los dibuja

como *gauges* (la barra se pinta en tono de alarma cuando el valor llega al 80 %). Así, cada métrica del cuadro proviene directamente del SO de la máquina que la reporta.

10. Generador de carga, pruebas y despliegue

Este capítulo cierra el manual con la cara operativa del sistema: cómo se le inyecta carga para medirlo, cómo se prueba su lógica con un arnés en Java puro y cómo se despliega y se opera el clúster sobre Google Cloud el día de la evaluación.

10.1. Generador de carga

El generador de carga vive en `loadtest/LoadDriver.java`, una clase *stand-alone* que es el único “cliente” del proyecto: no depende de nada del servidor más allá del contrato REST publicado. Genera tráfico contra un solo nodo (el líder) durante una duración fija mezclando un **80 % de lecturas de saldo y un 20 % de transferencias**, proporción gobernada por la constante `READ_RATIO = 0.80`. Por defecto corre `DEFAULT_SECONDS = 60` segundos con `DEFAULT_CLIENTS = 8` hilos concurrentes, pero los argumentos de `main` permiten fijar segundos, hilos y el rango de cuentas [`minId`, `maxId`]; subiendo los segundos puede correr de forma prácticamente indefinida, como exige la prueba del profesor que “nunca termina”.

Antes de medir, `bootstrapToken()` registra un usuario desechable (`loaddriver`) y hace `login` para obtener el JWT que comparten todos los hilos; tanto un 201 (creado) como un 409 (ya registrado) son aceptables porque solo se necesita poder iniciar sesión. Un detalle de rendimiento importante es que *un* único `HttpClient` (`sharedClient`) se reparte entre todos los *workers*: un cliente por hilo abriría cientos de hilos selectores propios y el generador se ahogaría en su propio *context-switch* antes de saturar al banco.

Cada hilo es un `Worker` que, hasta una fecha límite común en `System.nanoTime()`, sortea la operación según `READ_RATIO` y la despacha. Solo se cuentan los éxitos reales: `doRead()` incrementa `reads` únicamente si el servidor devolvió HTTP 200 con un saldo parseable, y `doTransfer()` incrementa `transfers` solo si la transferencia fue aceptada (también 200).

```
1 public void run() {
2     HttpClient http = sharedClient;
3     try {
4         while (System.nanoTime() < deadlineNanos) {
5             if (ThreadLocalRandom.current().nextDouble() < READ_RATIO) {
6                 doRead(http);
7             } else {
8                 doTransfer(http);
9             }
10        }
11    } finally {
12        done.countDown();
13    }
14 }
```

Listing 42: Bucle de carga por hilo (`Worker.run`, recortado)

La parte didácticamente más valiosa es la verificación de la invariante de conservación del dinero. Antes de arrancar la carga, `captureTotalCents()` lee `GET /api/stats` una sola vez y guarda `totalBalance` (en centavos): leer el total que el servidor suma en memoria es $O(1)$ en el cliente, en lugar de sumar cientos de miles de saldos por HTTP. Al terminar, `verifyAndReport()` vuelve a leer el total y exige que sea **idéntico** al inicial: ninguna transferencia crea ni destruye dinero, así que la suma global debe permanecer constante. Si no coincide, `reportCulprit()` recorre el rango buscando la cuenta con saldo negativo para señalar al culpable; si coincide, imprime `CONSISTENT` junto con las lecturas, las transferencias y el `score = transferencias*4 + lecturas` (una transferencia vale cuatro veces una lectura, según el PDF del concurso).

```

1 void verifyAndReport(long initialTotalCents, String scenario) {
2     long finalTotalCents = captureTotalCents();
3     if (finalTotalCents != initialTotalCents) {
4         System.out.println("INCONSISTENT: expected " + initialTotalCents
5             + " cents, found " + finalTotalCents + " cents");
6         reportCulprit(initialTotalCents, finalTotalCents);
7         return;
8     }
9     long score = transfers.get() * 4 + reads.get();
10    System.out.println("CONSISTENT");
11    // ...
12 }

```

Listing 43: Verificación de la invariante de conservación (verifyAndReport, recortado)

El cuerpo de la transferencia respeta la forma de cable obligatoria: `transferBody()` serializa `sourceAccountId` y `targetAccountId` como cadenas y `amount` como decimal; los montos salen de `randomAmount()` en el rango [0.01, 100.00], valores bajos a propósito para que las transferencias rara vez agoten una cuenta y el tráfico no degenera en puros rechazos por saldo insuficiente.

10.2. Harness de pruebas

El proyecto no usa JUnit: trae su propio arnés de pruebas en Java puro, formado por dos piezas. `tests/Assert.java` es un ayudante mínimo de aserciones (`isTrue`, `equals`, `notNull`, `throwsException`) donde cada método lanza un `AssertionError` con un mensaje descriptivo cuando la expectativa falla. `tests/RunTests.java` es el punto de entrada: instancia cada *suite*, ejecuta sus casos y, capturando el `AssertionError`, marca cada caso como PASS o FAIL sin detener a los demás. Un caso es un record `Case(String name, Runnable body)`, de modo que la falla se expresa simplemente lanzando una excepción desde el cuerpo. Al final imprime un resumen `total/passed/failed` y termina con estatus distinto de cero si algo falló, de manera que sirve para frenar la compilación.

```

1 List<Case> cases = new ArrayList<>();
2 cases.addAll(new LedgerTest().cases());
3 cases.addAll(new MoneyTest().cases());
4 cases.addAll(new RequestParserTest().cases());
5 cases.addAll(new WireCodecTest().cases());
6 cases.addAll(new AuthTest().cases());
7 cases.addAll(new ReplayTest().cases());
8 cases.addAll(new CheckpointTest().cases());
9 cases.addAll(new JwtCrossNodeTest().cases());
10 cases.addAll(new CatchupTest().cases());
11 // ... corre cada caso, cuenta PASS/FAIL y hace System.exit(1) si hubo fallos

```

Listing 44: Recolección y ejecución de suites (RunTests.main, recortado)

Las *suites* están organizadas por subsistema. `tests/MoneyTest.java` verifica que el dinero se redondee y convierta correctamente entre cadena decimal y centavos (con `HALF_UP`) y que el viaje de ida y vuelta sea exacto. `tests/LedgerTest.java` comprueba que `Ledger.move` transfiera centavos exactos entre dos cuentas y mantenga el total constante bajo concurrencia. `tests/RequestParserTest.java` valida que el parser arme una petición tanto en una sola entrega como partida entre varias. `tests/WireCodecTest.java` cubre la codificación de una transferencia a una línea JSON y su *round-trip*. `tests/AuthTest.java` prueba la autorización del `Authenticator`: el esquema `Bearer` es insensible a mayúsculas (RFC 7235) y tolera espacios, mientras que un encabezado ausente, un esquema equivocado o un token vacío o falso se rechazan.

El bloque de mayor valor para este sistema distribuido son las pruebas de recuperación y de réplica. `tests/ReplayTest.java` fija el contrato de `Bank.applyReplicated()`: un seguidor reproduce en orden de secuencia y debe *forzar* la aplicación de cada transferencia (ya fue autorizada por el líder) en vez de re-verificar fondos, porque re-verificar podría rechazar y *descartar* una transferencia autorizada mientras los contadores avanzan, divergiendo de forma permanente; aplicar incondicionalmente es independiente del orden y por tanto convergente. La *suite* alimenta una secuencia en orden adverso, comprueba la convergencia y que un reenvío de una secuencia ya vista sea idempotente. `tests/CheckpointTest.java` cubre las primitivas de *checkpoint* que permiten a una réplica reanudar desde su secuencia exacta tras un reinicio total: `Bank.snapshotInto` (captura atómica de marca de agua y saldos), `Bank.seedLastSeq` (siembra la marca desde un *checkpoint* restaurado) y la garantía de que un *snapshot* restaurado más el delta del líder converge exactamente a los saldos por cuenta del líder. `tests/CatchupTest.java` fija la reanudación por secuencia exacta: que `log.since(X)` sea justamente la cola `X+1..` y que un banco sembrado en la marca `X` descarte los reenvíos `<=X` y aplique desde `X+1`. `tests/JwtCrossNodeTest.java` fija la propiedad entre nodos del JWT: un token emitido con el secreto compartido valida en otro nodo y uno firmado con un secreto distinto se rechaza.

Existen además dos verificaciones de integración que **no** corre `RunTests` porque abren un socket real: `tests/ReplConsistency.java` cablea un `Bank` líder con uno réplica sobre `ReplicaFeed` y `ReplicaSync`, lo martilla con varios hilos escritores haciendo miles de transferencias concurrentes y luego exige que el saldo **por cuenta** líder = réplica para cada cuenta (la suma global se conserva aun si una transferencia se pierde o se duplica, así que solo la comparación por cuenta atrapa un *bug* de replicación). `tests/ReplInversion.java` es la regresión de la convergencia por cuenta bajo carga concurrente con saldos deliberadamente ajustados y montos de hasta el saldo completo – la ruta que originalmente expuso el *bug* de inversión de secuencia –, que `ReplConsistency` no ejercita porque sus cuentas arrancan demasiado holgadas. Cada una imprime `REPL_CONSISTENCY_OK` o `REPL_INVERSION_OK` y sale con 0 en éxito; se ejecutan a mano con el *fat jar* en el *classpath*.

10.3. Despliegue en GCP

La infraestructura se aprovisiona con `deploy/crear-infra.sh`, que crea en Google Cloud (zona `us-central1-a`) una IP estática para el líder, las reglas de *firewall* (puerto 8080 abierto a clientes y *dashboard*, 9090 solo entre nodos con la etiqueta `tesoreria`) y exactamente **2x** `e2-standard-2` + **1x** `e2-standard-4`, respetando la restricción de usar solo la serie E2. Los tres nodos arrancan el *mismo* `deploy/startup-script.sh` y difieren únicamente por metadatos por nodo: `tes-node-id`, `tes-leader-host`, `tes-peers` y un `tes-jwt-secret` compartido. La convención clave es que un `tes-leader-host` *vacío* marca al nodo como líder (que recibe la IP estática); las réplicas lo siguen por su nombre vía el DNS interno de la VPC.

```

1 # Leader: empty tes-leader-host marks it the leader; it gets the static IP.
2 create_node "${LEADER}" "e2-standard-4" "" "${REPLICA_A}:${HTTP_PORT},${REPLICA_B}:${HTTP_PORT}" \
3   --address="${LEADER_IP}"
4 # Replicas: follow the leader by name; e2-standard-2 each.
5 create_node "${REPLICA_A}" "e2-standard-2" "${LEADER}" "${LEADER}:${HTTP_PORT},${REPLICA_B}:${HTTP_PORT}"
6 create_node "${REPLICA_B}" "e2-standard-2" "${LEADER}" "${LEADER}:${HTTP_PORT},${REPLICA_A}:${HTTP_PORT}"

```

Listing 45: Creacion del cluster: 1 lider e2-standard-4 + 2 replicas e2-standard-2 (crear-infra.sh, recortado)

En cada arranque, `startup-script.sh` corre como *root* y hace cuatro cosas: instala el *runtime* `openjdk-17-jre-headless`; descarga desde Cloud Storage el *fat jar*, el *dataset* `alumnos.csv`, la unidad `node.service` y la llave `gcs-key.json`; escribe el archivo de entorno `/etc/tesoreria/node.env`

con las variables `TES_*` leídas de los metadatos (entre ellas `TES_NODE_ID`, `TES_PEERS`, `TES_LEADER_HOST`, `TES_JWT_SECRET` y `TES_BUCKET`); e instala y arranca la unidad *systemd*. Que el mismo script sirva para todos los roles es lo que mantiene la operación simple: la diferencia líder/réplica se infiere de `TES_LEADER_HOST`.

La unidad `deploy/node.service` lanza el *jar* con el puerto HTTP como primer argumento (`ExecStart=/usr/bin/java -jar ../tesoreria-distribuida.jar 8080`), lee el entorno de `/etc/tesoreria/node.env` y reinicia on-failure para que un nodo caído reingrese al clúster automáticamente. Lo **más importante** de esta unidad es su política de apagado: usa `KillSignal=SIGTERM` con `TimeoutStopSec=120` para que la JVM ejecute sus *shutdown hooks*. Un apagado *graceful* con `SIGTERM` dispara, en una réplica, la escritura del *checkpoint* exacto (de ahí depende la garantía de reanudar desde la secuencia precisa) y, en el líder, el drenado de la cola del *journal* durable. Por eso conviene distinguir reiniciar el *proceso* de reiniciar la *VM*: matar el proceso con `SIGTERM` (o `systemctl stop`) preserva el estado y el *checkpoint*; nunca debe usarse `kill -9`, que cortaría la escritura del *checkpoint* a media subida.

```
1 KillSignal=SIGTERM
2 TimeoutStopSec=120
```

Listing 46: Apagado graceful que preserva el checkpoint (node.service, recortado)

El ciclo de vida del despliegue se cubre con scripts cortos para no gastar cómputo cuando no se usa: la infraestructura normalmente está apagada y el día de la revisión basta `deploy/encender.sh` para arrancar las cuatro VMs (los tres nodos más el **generador**) y esperar a que el líder cargue las 820k cuentas y sirva `/api/stats`, sondeándolo con reintentos de `curl`. Al terminar, `deploy/apagar.sh` detiene las VMs sin borrarlas (pausa el gasto) y `deploy/destruir-infra.sh` elimina todo, incluidos discos e IP estática, para no dejar ningún costo. La IP estática del líder no cambia al apagar y encender, así que el punto de entrada único es estable.

10.4. Operacion el dia del concurso

El guion de la demo en vivo está en `deploy/RUNBOOK.md`, organizado como **encender** → **demostrar** → **apagar**. El paso 0 levanta el clúster con `encender.sh` y abre el *dashboard* en `http://34.67.240.245:8080/` (la IP estática del líder). El paso 1 ejercita a mano los cuatro *end-points* con JWT (`/api/register`, `/api/login`, `/api/accounts/{id}` y `/api/transactions/transfer`) para mostrar sus formas exactas, y el paso 2 demuestra la invariante de conservación: `totalBalance` en `/api/stats` debe quedar constante en 410113948069.86 antes y después de transferir.

El corazón de la evaluación es la tolerancia a fallos (pasos 3 y 4). Se demuestra el “mata-proceso” apagando primero `nodo-3` y luego `nodo-2` con `gcloud compute instances stop`: el líder **sigue sirviendo** aunque caigan las dos réplicas. Después se revive una réplica y se observa en su *log* que reanuda desde su secuencia *exacta* (no desde cero), gracias al *checkpoint* que escribió al apagarse con `SIGTERM`, y que converge al líder (mismo `lastTxId` y mismo `totalBalance`). El *dashboard* muestra todo el tiempo, por nodo, su estado, #cuentas, saldo total, #transferencias, última tx y uso de CPU/RAM/disco, conservando los saldos.

```
1 $GC compute instances start nodo-2 --zone=$ZONE      # revive
2 # en su log se ve que reanuda desde su secuencia EXACTA (no desde 0):
3 #   -> [checkpoint] restored checkpoint/nodo-2.json at watermark <N>
4 #   -> [node] replica resuming at sequence <N>
```

Listing 47: Revivir una replica y verificar reanudacion por secuencia (RUNBOOK paso 4, recortado)

La medición del concurso se automatiza con `deploy/concurso.sh`, que corre el `LoadDriver` desde la **VM generador** (baja latencia contra la IP interna del líder) en tres escenarios encadenados, apagando una réplica entre uno y otro: escenario 1 con el clúster completo (3 nodos),

escenario 2 con líder + 1 réplica y escenario 3 solo con el líder. Cada escenario verifica la invariante (**CONSISTENT**) e imprime lecturas, transferencias y su **score = transferencias*4 + lecturas**; el script suma los tres *scores* en un total. Que las tres corridas terminen **CONSISTENT** aun cruzando apagados de réplicas es la prueba operativa de que el clúster no pierde ni inventa dinero bajo fallos.

```
1 out="$(SSH_GEN "java -cp /opt/tesoreria-distribuida.jar mx.ipn.escom.tesoreria.  
    loadtest.LoadDriver ${LEADER_INT} 8080 ${SECONDS_RUN} ${THREADS} 1 820000 ${  
    label} 2>&1")"  
2 echo "${out}" | grep -E 'CONSISTENT|INCONSISTENT|successful|score' >&2
```

Listing 48: Un escenario del concurso sobre el generador (concurso.sh, recortado)

Para cerrar, el RUNBOOK documenta la recuperación ante desastre total: como el estado durable vive en Cloud Storage, un arranque en frío de los nodos (incluso tras destruir las VMs y volver a crearlas) reconstruye el banco desde el bucket. La limpieza final del día es `deploy/apagar.sh` para solo pausar el gasto, o `deploy/destruir-infra.sh` para borrar discos e IP estática y no dejar ningún costo. El RUNBOOK advierte además que las réplicas siempre se apaguen con `SIGTERM` y nunca con `kill -9`, para no truncar la escritura del *checkpoint*.